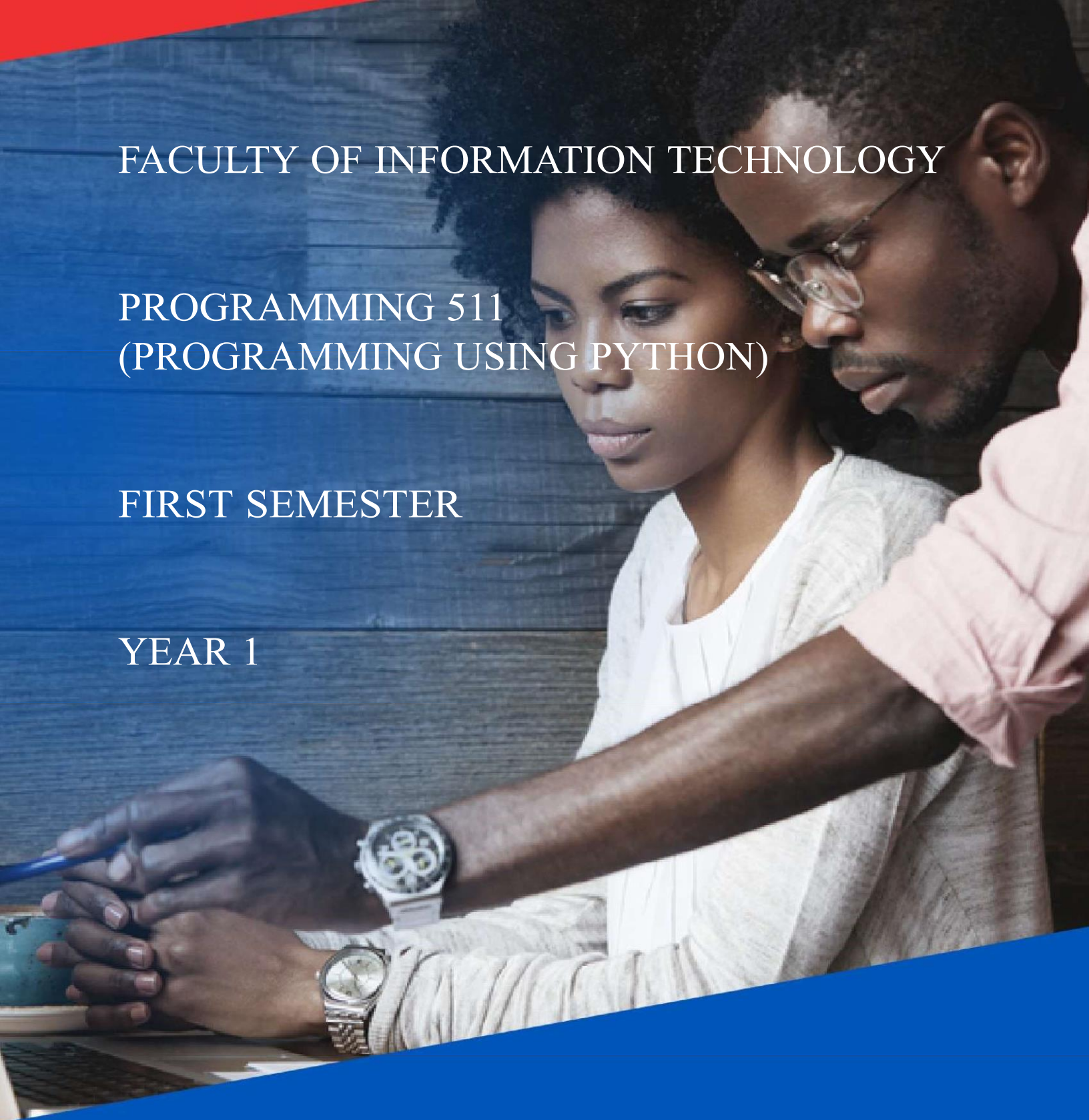


FACULTY OF INFORMATION TECHNOLOGY

PROGRAMMING 511
(PROGRAMMING USING PYTHON)

FIRST SEMESTER

YEAR 1

RICHFIELD

richfield.ac.za



FACULTY OF INFORMATION TECHNOLOGY

STUDY GUIDE

MODULE: PROGRAMMING 511 (1ST SEMESTER)

Copyright © 2024
Richfield Graduate Institute of Technology (Pty) Ltd
Registration Number: 2000/000757/07
All rights reserved; no part of this publication may be reproduced in any form or by any means, including photocopying machines, without the written permission of the Institution

Table of Contents

SECTION A: PREFACE 1

INTRODUCTION 1

Purpose of the Module 1

MODULE LEARNING OUTCOMES 2

METHOD OF STUDY 2

LECTURES AND TUTORIALS 3

PRESCRIBED & RECOMMENDED MATERIAL 3

LIBRARY INFRASTRUCTURE 4

ASSESSMENT 4

Key Concepts in Assignments and Examinations 5

WORK READINESS PROGRAMME (WRP) 7

WORK INTEGRATED LEARNING 8

TOPIC 1 10

INTRODUCTION TO COMPUTERS AND PROGRAMMING 10

Learning Outcomes: 10

1.1 Introduction ..... **Error! Bookmark not defined.**

1.2 Hardware and Software Hardware ..... **Error! Bookmark not defined.**

1.3 How Computers Store Data ..... **Error! Bookmark not defined.**

1.4 How a Program Works ..... **Error! Bookmark not defined.**

1.5 Using Python ..... **Error! Bookmark not defined.**

Review Questions 26

TOPIC 2 29

PYTHON FUNDAMENTALS 29

Learning Outcomes 29

2.1 Introduction 29

2.2 Designing a Program ..... **Error! Bookmark not defined.**

2.3 Input, Processing, and Output ..... **Error! Bookmark not defined.**

2.4 Displaying Output with the print Statement ..... **Error! Bookmark not defined.**

2.5 Comments ..... **Error! Bookmark not defined.**

2.6 Variables ..... **Error! Bookmark not defined.**

2.7 Performing Calculations 42

2.8 Escape Characters and String Concatenation 46

2.9 Formatting 47

2.10 Named Constants 50

Review Questions 51

TOPIC 3 54

DECISION STRUCTURES AND BOOLEAN LOGIC 54

Learning Outcomes 54

3.1 Introduction 54

3.2 The if Statement 54

3.3 The if-else Statement 58

3.4 Comparing Strings 60

3.5 Nested Decision Structures and the if-elif-else Statement 62

3.6 Logical Operators 64
3.7 Boolean Variables 67
Review Questions 69
TOPIC 4 72
REPETITION STRUCTURES 72
Learning Outcomes: 72
4.1 Introduction 72
4.2 Introduction to Repetition Structures 72
4.3 The while Loop: a Condition-Controlled Loop 72
4.4 The for Loop 75
4.5 Calculating a Running Total 79
4.6 The Augmented Assignment Operators 80
4.7 Sentinels 80
4.9 Nested Loops 82
Review Questions 84
TOPIC 5 86
FUNCTIONS 86
Learning Outcomes: 86
5.1 Introduction 86
5.2 Defining and Calling a Function 88
5.4 Local Variables 93
5.5 Passing Arguments to Functions 94

5.6 Global Variables and Global Constants 100

5.7 Introduction to Value-Returning Functions 102

5.8 The math Module 108

Review Questions 111

TOPIC 6 113

FILES 113

Learning Outcomes: 113

6.1 Introduction 113

6.1 Types of Files 113

6.2 Opening a File 114

6.3 Writing Data to a File 115

6.4 Reading Data From a File 116

6.5 Closing a file 116

6.6 Concatenating a Newline to a String 119

6.7 Appending Data to an Existing File 122

6.8 Writing and Reading Numeric Data 123

6.9 Reading a File with a Loop and Detecting the End of the File 124

Review Questions 126

References 128

SECTION A: PREFACE

INTRODUCTION

Welcome to the Faculty of Information Technology at Richfield Graduate Institute of Technology.

We trust you will find the contents and learning outcomes of this module both interesting and insightful as you begin your academic journey and eventually, your career in the IT world. This section of the study guide is intended to orientate you to the module before the commencement of formal lectures. The following lectures will focus on the study units described

Module details

Table 1: Module details

1nd Semester	Details
Title Of Module: NQF Level: Credits: Mode of Delivery:	Programming 511 NQF 5 15 Contact/Distance

Purpose of the Module

The purpose of this module is to equip students with the skills required to write computer programs that solve real-world problems. The assumption in this module's design is that students have no prior programming experience and will pick up concepts through hands-on training. One of the most popular programming languages, Python is used to create video games, statistical models, and algorithms for artificial intelligence, among a plethora of other things. Students will learn the basic and advanced programming concepts that will serve as a basis for all other programming modules that are in the curriculum.

Programming 511

This module exposes learners to the various aspects of programming, including understanding the fundamentals of data storage, input and output, functions, sequences and lists, file I/O, decision structures and boolean logic, repetition structures, functions, files and exceptions,

lists and tuples, dictionaries and sets, classes, object-oriented programming, inheritance and objects that are created from standard library classes.

MODULE LEARNING OUTCOMES

On completion of this module, students should have a basic/fundamental practical and theoretical knowledge of:

- How a program works
- Writing programs using the correct Python syntax
- Create programs that use variables of different datatypes
- Using decision structures and boolean logic to write programs whose output depends on certain conditions
- Repetition structures to implement programs with statements that are executed numerous times.
- Modularising a program using functions
- File operations used to create new files; write and append text to them; and read text from them
- User containers such as lists and tuples to store collections of objects; and perform operations on them.

METHOD OF STUDY

Only the key sections that have to be studied are indicated under each topic in this study guide are expected to have a thorough working knowledge of the prescribed textbook. These form the basis for tests, assignments and examinations. To be able to do the activities and assignments for this module, and to achieve the learning outcomes and ultimately to be successful in the tests and exams.

You will need an in-depth understanding of the content of these sections in the learning guide and the prescribed books. To master the learning material, you must accept responsibility for your studies. Learning is not the same as memorising. You are expected to show that you understand and can apply the information. Lectures, tutorials, case studies and group discussions may also be used to present this module.

LECTURES AND TUTORIALS

Students must refer to the notice boards on their respective campuses for details of the lecture and tutorial timetables. The lecturer assigned to the module will also inform you of the number of lecture periods and tutorials allocated to a particular module. Prior preparation is required for each lecture and tutorial. Students are encouraged to actively participate in lectures and tutorials to ensure success in tests, group discussions, practicals, assignments and examinations.

NOTICES

All information about this module such as tests dates, lecture and tutorial timetables, assignments, examinations etc. will be displayed on the notice board located at your campus. Students must check the notice board daily. Should you require any clarification, please consult your lecturer, programme manager or administrator of your respective campus.

PRESCRIBED & RECOMMENDED MATERIAL

PRESCRIBED MATERIAL

Gaddis, T. (2021). Starting out with Python. 5th Global Ed. United Kingdom: Pearson Education. ISBN: 9781292408637

RECOMMENDED MATERIAL

Python.org. (2019). The Python Tutorial — Python 3.8.0 documentation. [online] Available at: https://docs.python.org/3/tutorial/.

w3Schools (2019). Introduction to Python. [online] W3schools.com. Available at: https://www.w3schools.com/python/python_intro.asp.

Learnpython.org. (2019). Learn Python - Free Interactive Python Tutorial. [online] Available at: https://www.learnpython.org/.

LIBRARY INFRASTRUCTURE

The following services are available to you:

- Each campus keeps a limited quantity of the recommended reading titles and a wider variety of similar titles which you may borrow. Please note that students are required to purchase the prescribed materials.
- Arrangements have been made with municipal, state and other libraries to stock our recommended reading and similar titles. You may use these on their premises or borrow them if available. It is your responsibility to safe keeps all library books.
- RGIT has also allocated one library period per week to assist you with your formal research under professional supervision.
- RGIT has dedicated electronic libraries for use by its students. The computers laboratories, when not in use for academic purposes, may also be used for research purposes. Booking is essential for all electronic library usage.

ASSESSMENT

The assessment for this module will comprise two Continuous Assessment (CA) Tests, an assignment and an examination. Your lecturer will inform you of the dates, times and the venues for each of these. You may also refer to the notice board on your campus or the Academic Calendar, which is displayed in all lecture rooms.

CONTINUOUS ASSESSMENT TESTS

ASSIGNMENT

There is one compulsory assignment for each module in each semester. Your lecturer will inform you of the Assignment questions at the commencement of this module.

EXAMINATION

There is one three-hour examination for each module. Make sure that you diarise the correct date, time and venue. The examinations department will notify you of your results once all administrative matters are cleared, and fees are paid up. The examination may consist of multiple-choice questions, short questions and essay type questions.

This requires you to be thoroughly prepared as all the content matter of lectures, tutorials, all references to the prescribed text and any other additional documentation/reference materials are examinable in both your tests and the examinations. The examination department will

make available to you the details of the examination (date, time and venue) in due course. You must be seated in the examination room 15 minutes before the commencement of the examination. If you arrive late, you will not be allowed any extra time. Your learner registration card must always be in your possession.

FINAL ASSESSMENT

There are two compulsory tests for each module (in each semester). The final assessment for this module will be weighted as follows:

CA Test 1	13%
CA Test 2	13%
Assignment	14%
	(Total Mark: 40%)
Examination	60%
Total	100%

Key Concepts in Assignments and Examinations

In assignment and examination questions, you will notice certain vital concepts (i.e. words/verbs) which tell you what is expected of you. For example, you may be asked in a question to list, describe, illustrate, demonstrate, compare, construct, relate, criticise, recommend or design information/aspects/factors/situations. To help you to know what these key concepts or verbs mean so that you will know what is expected of you, we present the following taxonomy by Bloom, explaining the concepts and stating the level of cognitive thinking that theses refer to.

Table 2: Bloom's Taxonomy

Competence	Skills Demonstrated
Knowledge	• Observation and recall of information • Knowledge of dates, events, places • Knowledge of major ideas • Mastery of subject matter Question Cues list, define, tell, describe, identify, show, label, collect, examine, tabulate, quote, name, who, when, where, etc.
Comprehension	• Understanding information • Grasp meaning • Translate knowledge into a new context • Interpret facts, compare, contrast • Order, group, infer causes • Predict consequences Question Cues summarize, describe, interpret, contrast, predict, associate, distinguish, estimate, differentiate, discuss, extend
Application	• Use information • Use methods, concepts, theories in new situations • Solve problems using required skills or knowledge Questions Cues apply, demonstrate, calculate, complete, illustrate, show, solve, examine, modify, relate, change, classify, experiment, discover

Analysis	• Seeing patterns • Organisation of parts • Recognition of hidden meanings • Identification of components Question Cues analyse, separate, order, arrange, divide, compare, select, infer, connect, classify, explain
Synthesis	• Use old ideas to create new ones • Generalise from given facts Relate knowledge from several areas Predict, draw conclusions Question Cues combine, integrate, modify, rearrange, substitute, plan, create, design, invent, what if? compose, formulate, prepare, generalise, rewrite
Evaluation	• Compare and discriminate between ideas • Assess the value of theories, presentations • Make choices based on reasoned argument Verify value of evidence recognise subjectivity Question Cues assess, decide, rank, grade, test, measure, recommend, convince, select, judge, explain, discriminate, support, conclude, compare, summarise

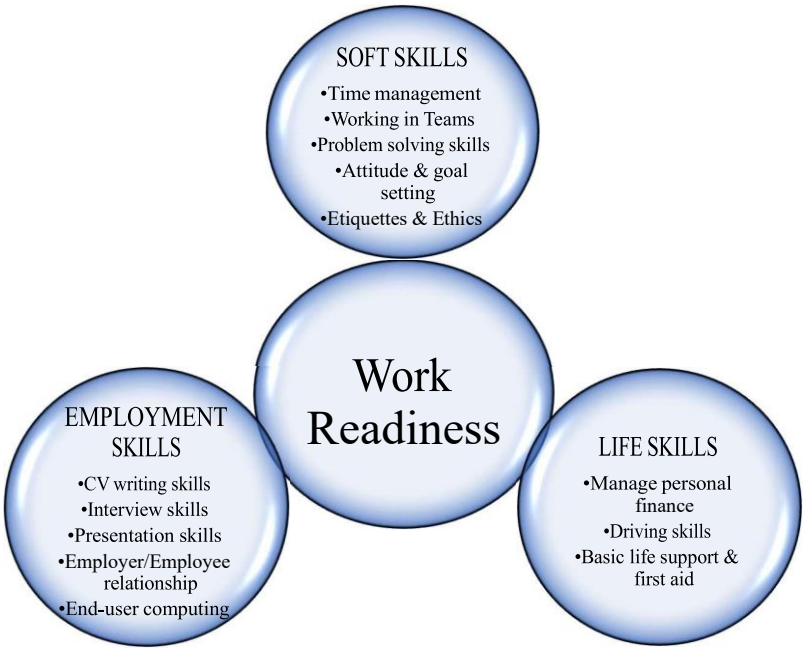
WORK READINESS PROGRAMME (WRP)

To prepare students for the world of work, a series of interventions over and above the formal curriculum, are concurrently implemented to prepare students. These include:

- Soft skills
- Employment skills
- Life skills
- End-User Computing (if not included in your curriculum)

It is in your interest to attend these workshops, complete the Work Readiness Logbook and prepare for the working world. The illustration below outlines some of the key concepts for Work Readiness that will be included in your timetable.

Figure 1: Work Readiness



WORK INTEGRATED LEARNING

Work Integrated Learning (WIL) forms a core component of the curriculum for the completion of this programme. All modules which form part of this qualification will be assessed in an integrated manner towards the end of the programme or after completion of all other modules. Prerequisites for placement with employers will include:

- Completion of all tests & assignment
- Success in examination Payment of all arrear fees
- Return of library books, etc.
- Completion of the Work Readiness Programme (WRP).

Students will be fully inducted on the Work Integrated Learning Module, the Workbooks & assessment requirements before placement with employers. The partners in the Work Integrated Learning are the same as the Work Readiness Programme:

Figure 1: Work Integrated Learning



Good luck and success in your studies…

TOPIC 1
INTRODUCTION TO COMPUTERS AND PROGRAMMING



Learning Outcomes:

After studying this topic you should be able to:

- Understand the fundamental workings of computers.
- Comprehend how data is stored and manipulated in computer systems.
- Recognize the advantages of using high-level programming languages.
- Write basic programs using Python's syntax and structure.

1.1 Introduction

The first section of this topic provides an insight into the functioning of computers, data storage and manipulation, and the rationale behind using high-level languages for programming. Key concepts such as Python programming, interactive and script modes, and the IDLE (Integrated Development and Learning Environment) are introduced.

Understanding How Computers Work

Computers are integral to our daily lives, found in various forms such as desktops, laptops, smartphones, and embedded systems in appliances and vehicles. Their versatility stems from the ability to be programmed to perform a myriad of tasks. At the core, a computer is a machine that processes data and executes instructions.

Data Storage and Manipulation

- **Data Representation**: Computers use binary (0s and 1s) to represent data. This binary system is the basis of data storage and manipulation in computer systems.
- **Data Processing**: Through a combination of hardware and software, computers can process this binary data to perform calculations, data analysis, and various other tasks.

High-Level Programming Languages

- **Why High-Level Languages?**: These languages, such as Python, are closer to human language, making them easier to learn and use. They abstract the complexities of binary code and hardware specifics, enabling programmers to focus on solving problems.
- **Advantages**: Improved readability, easier debugging and maintenance, and cross-platform compatibility.

Introduction to Python

- **Python as a Choice**: It is an ideal programming language for beginners due to its simplicity and readability. Python is versatile, widely used in web development, data science, artificial intelligence, and more.
- **Interactive Mode vs Script Mode**: Python can be used in two modes:
    - **Interactive Mode**: Allows you to type Python commands and get immediate feedback. Ideal for learning and experimenting.
    - **Script Mode**: Involves writing Python code in a file and then running it as a program. Used for developing complete applications.
- **IDLE Environment**: IDLE is Python's Integrated Development and Learning Environment, which provides a user-friendly interface to write, debug, and execute Python scripts.

Practical Application

- **Writing Basic Programs**: Start with simple tasks like printing text, performing arithmetic operations, and gradually progress to more complex programming constructs like loops, functions, and data structures in Python.

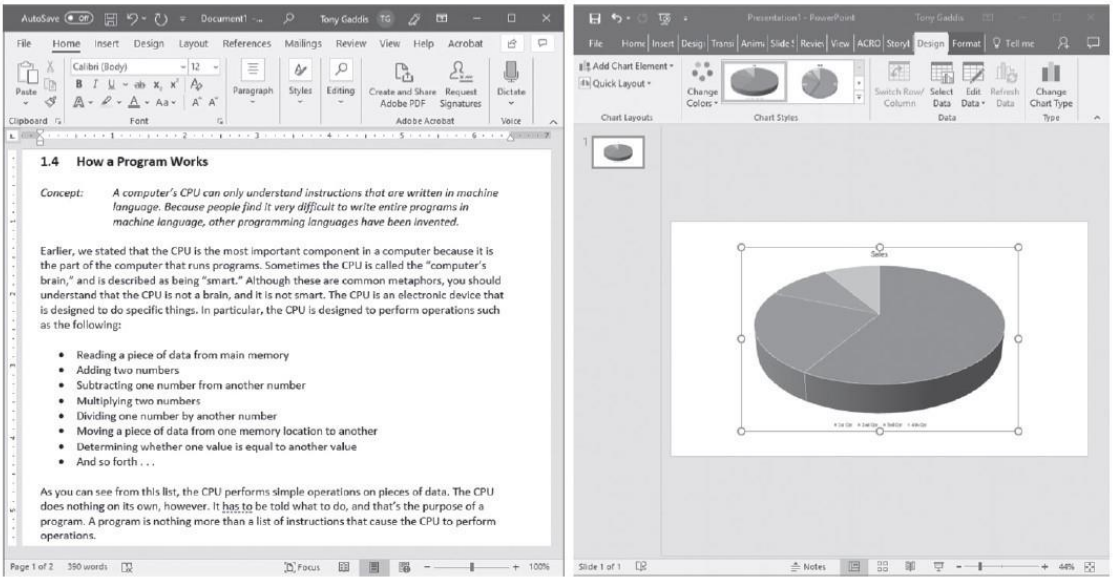


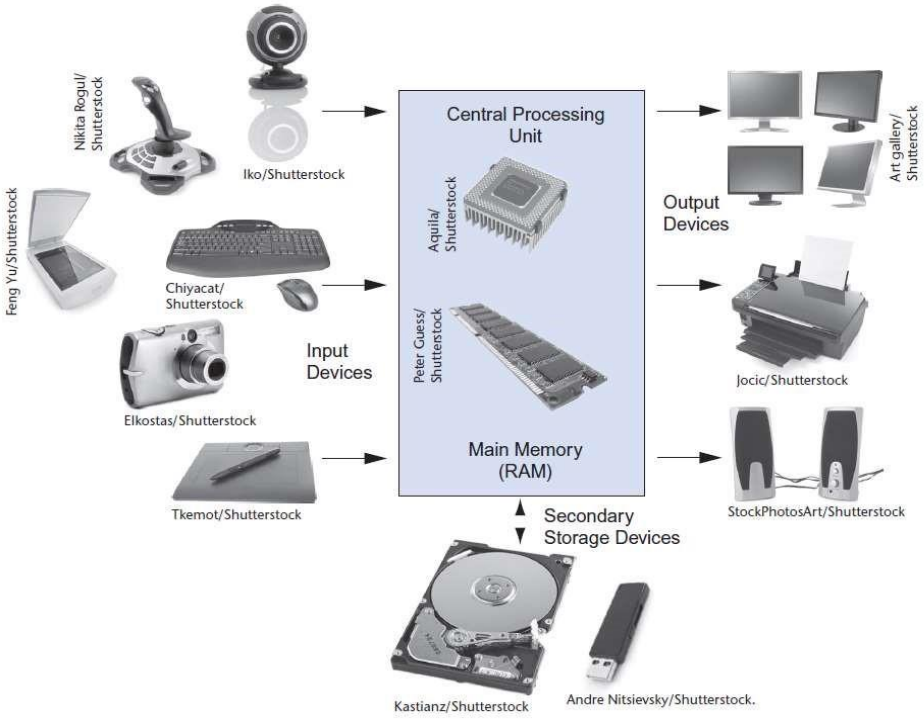
Figure 1-1 A word processing program and a presentation program (Gaddis, 2021)

1.2 Hardware and Software

1.2.1 Understanding Computer Hardware

Introduction to Computer Systems

A computer is a sophisticated device composed of various hardware components, each performing specific functions to facilitate the execution of tasks. Understanding these components is crucial for grasping how computers operate and how they execute programming instructions, such as those written in Python.



Key Components of a Computer System

1. **Central Processing Unit (CPU)**

    - **Definition**: The CPU, often referred to as the brain of the computer, is responsible for executing program instructions. It performs the basic arithmetical, logical, and input/output operations of the system.
    - **Function in Programming**: In Python programming, the CPU executes the interpreted Python code, line by line. Its efficiency directly affects the speed at which a Python program runs.
    - **Historical Context**: An example of an early CPU is found in the ENIAC, the first programmable computer built in 1945. Despite its primitive design by modern standards, with a height of 8 feet, a length of 100 feet, and a weight of 30 tons, it laid the foundation for modern computing

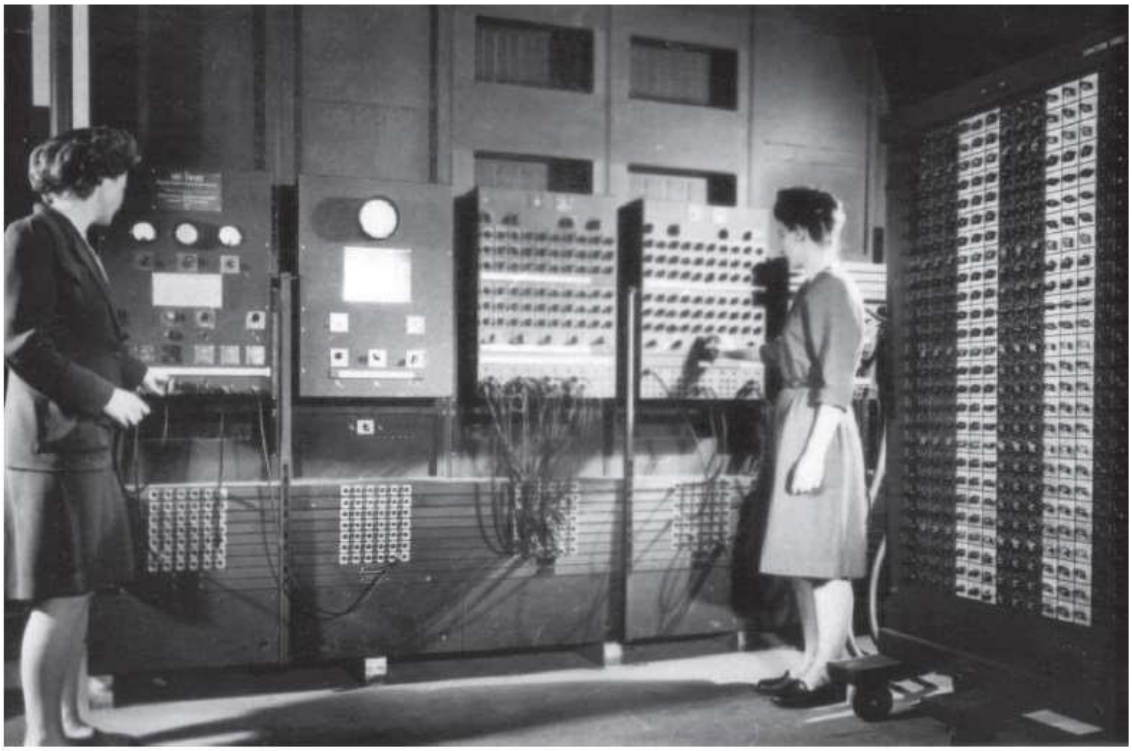


Figure 1-3 The ENIAC computer (Gaddis, 2021)

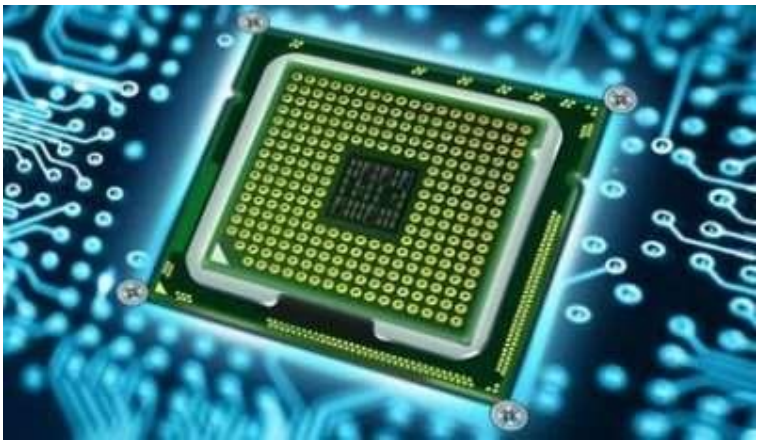


Figure 1-4 A lab technician holds a modern microprocessor

Microprocessor are much smaller and much more powerful.

1. **Main Memory (RAM)**
    - **Role in Computing**: Main memory, or Random Access Memory (RAM), is where a computer stores data temporarily. It holds data that is actively being used or processed by the CPU.
    - **Relevance to Python**: Python programs and their variables are stored in RAM during execution. The amount of RAM available can affect the performance and the capability of handling large data sets in Python applications.
2. **Secondary Storage**
    - **Purpose**: Secondary storage devices, such as hard drives and SSDs, store data persistently. Unlike RAM, this data is not erased when the computer is turned off.
    - **Impact on Python Programming**: Secondary storage is where Python programs and files are stored long-term. Python can read from and write to these storage devices, allowing for data persistence beyond the program's runtime.

1.2.2 Software Aspect

Python's Interaction with Hardware

- While Python itself is software, its performance and capabilities are inherently tied to the hardware it runs on. Efficient CPU processing, adequate RAM, and sufficient secondary storage are key for optimal Python program performance.



Think point: Can you think of any other input devices?

Output Devices

Output devices are essential components in computing, as they allow us to visualize and interact with the data processed by computers. These devices are crucial not only for general computer use but also in Python programming, where outputs are often displayed or printed for debugging, data analysis, or user interaction.

Common Output Devices:

- **Monitor:** Displays visual output from a computer. Python programs often use GUI libraries like Tkinter or PyQt to create applications whose results are displayed on monitors.
- **Printer:** Converts digital data into a physical format. Python can be used to format and send documents or reports to a printer.

1.2 Software

Software is the non-tangible component of computing. It's categorized into system software and application software, both of which are crucial in Python programming.

System Software:

- **Operating System:** Manages hardware and is essential for running Python interpreters or integrated development environments (IDEs) like PyCharm or Visual Studio Code.
- **Utility Programs:** These include software like antivirus programs and system management tools, which can be automated or enhanced using Python scripts.
- **Software Development Tools:** Python itself falls under this category as it's used to create, modify, and test software applications.

Application Software:

This is software used for performing specific tasks. Python is often used to develop various kinds of application software, like web applications (using frameworks like Django or Flask), data analysis tools (using libraries like Pandas and NumPy), and automation scripts.

Checkpoint: Think of other application software you use daily. How might Python play a role in these tools?

1.3 How Computers Store Data

Understanding data storage is crucial for Python programmers, especially when dealing with data types and memory management.

- **Bytes and Bits:** In Python, understanding bytes is important when dealing with binary data, file I/O, and network communications. Python has a **bytes** type for handling binary data.
- **Positive and Negative Charges:** The binary system (0s and 1s) in computing correlates to the positive and negative charges in bits. In Python, boolean values (**True** and **False**) are often used to represent these binary states.

Enhancing Python Learning:

Understanding these fundamentals is crucial for Python programmers, especially when dealing with file handling, data processing, and creating software applications. Recognizing the role of hardware and system software in running Python applications helps in optimizing performance and developing more efficient programs.

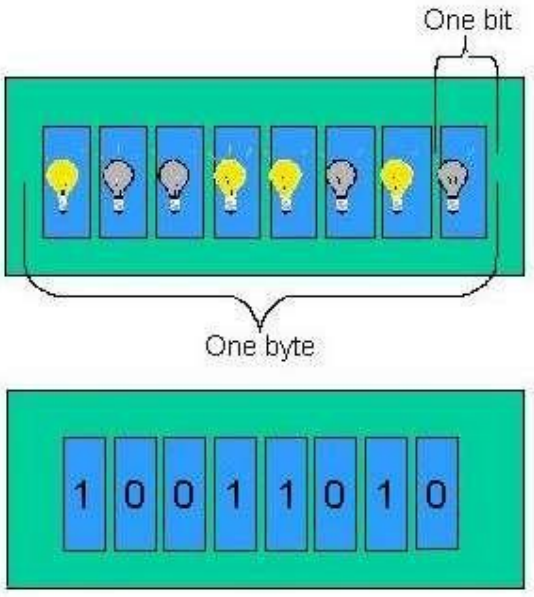


Figure 1-5 Think of a byte as eight switches

1.3. **Storing Numbers in Python**

1. Introduction to Binary System

- **Binary Basics**:
    - In digital electronics and computing, information is represented using two states: 'on' and 'off'.
    - These states correspond to two digits in the binary system: 1 (on) and 0 (off).
    - This system, known as binary, forms the basis of all modern computing systems.

2. Binary Numbering System

- **Structure**:
    - A binary number is composed exclusively of 0s and 1s. Each digit in this system is called a "bit".
    - The position of each bit in a binary number determines its value, increasing exponentially from right to left, starting at 0.
    - For example, in the binary number 1011, the rightmost bit (1) represents 2^0, the next bit (1) represents 2^1, and so on.

3. Decimal to Binary Conversion

- **Understanding Conversion**:
    - In Python, you can convert decimal numbers (base-10) to binary (base-2) using the **bin()** function.
    - Example: **bin(10)** will return **'0b1010'**, where '0b' is a prefix indicating a binary number.

4. Binary Arithmetic in Python

- **Operations**:
    - Python allows binary arithmetic, such as addition (+), subtraction (-), multiplication (*), and division (/).
    - Bitwise operations like AND (**&**), OR (|), XOR (^), NOT (~), left shift (<<), and right shift (>>) are also performed on binary numbers.

5. Practical Applications

- **Binary in Python Programming**:
    - Understanding binary is crucial for tasks like data encoding, file I/O operations, network communications, and system design in Python.
    - Python's ability to easily handle binary data makes it a powerful tool for developing applications that interact with low-level system components

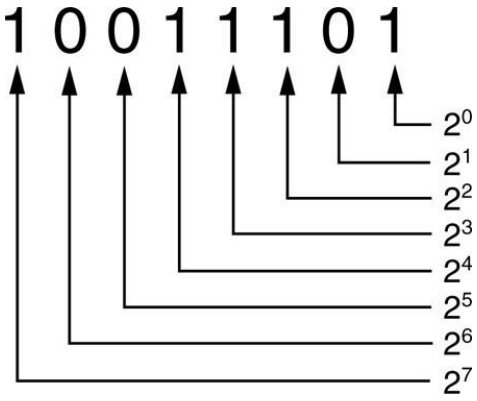
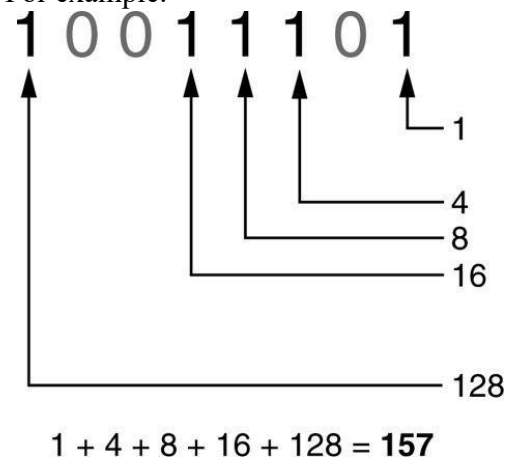
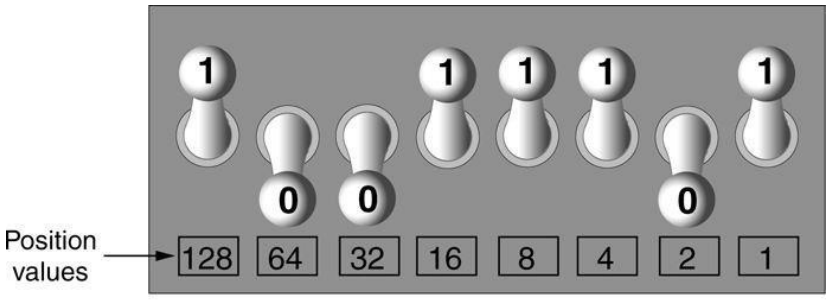


Figure 1-6 The values of binary digits as powers of 2

For example:



1 + 4 + 8 + 16 + 128 = **157**



128 + 16 + 8 + 4 + 1 = **157**

Figure 1-7 Determining the value of 10011101 (The bit pattern for 157)

Source: Gaddis (2021) page 9

The largest value that can be stored in a byte with eight bits is 255 and two bytes are used for larger numbers; maximum value is 65535

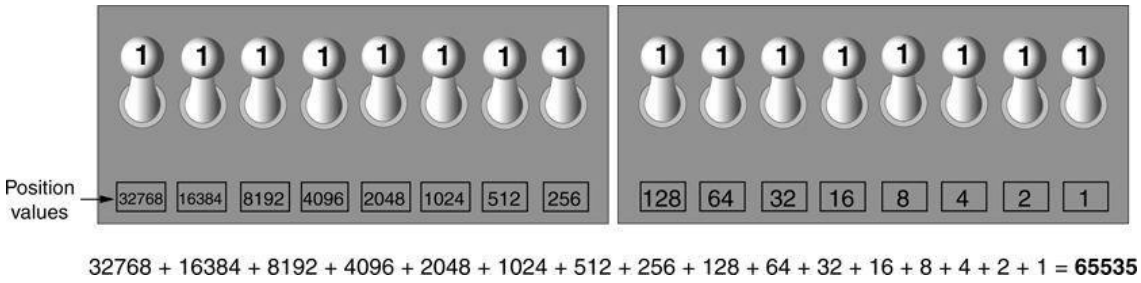


Figure 1-8 Two bytes used for a large number

1.3.2 **Storing Characters in Python**

Understanding Character Storage

- **Binary Representation**:
    - In computing, characters are stored as binary numbers, allowing for efficient processing and storage.
    - Each character, including letters, punctuation marks, and symbols, is mapped to a unique binary number.

ASCII: The Character Encoding Standard

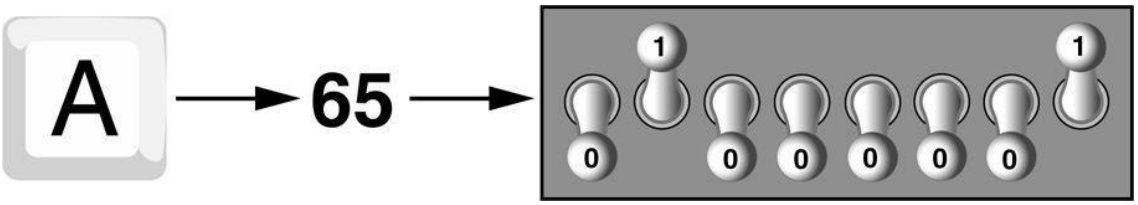
- **American Standard Code for Information Interchange (ASCII)**:
    - ASCII is a widely-used character encoding standard that represents text in computers and other devices.
    - It consists of a set of 128 numeric codes corresponding to English letters, punctuation, and control characters.
    - For example, the ASCII code for uppercase 'A' is 65, which in binary is **01000001**.

Python and Character Encoding

- **Manipulating ASCII in Python**:
    - Python allows you to convert characters to their ASCII values using the **ord()** function and vice versa using the **chr()** function.
    - Example: **ord('A')** returns 65, and **chr(65)** returns 'A'.

Practical Application and Exercises

- **Exercises for Understanding**:
    - Convert a string to its ASCII values and back.
    - Create a Python program to display the ASCII art using characters.
    - Understand how Python handles Unicode characters, which is an extension of ASCII allowing a vast array of global characters and symbols.



Your computer stores the number 65 when you input an uppercase A using the keyboard. This 65 is stored as a binary number.

Advanced Number Storage

- **Binary Number System**:
  - Primarily used for representing integer numbers.
  - Efficient in representing positive integers but requires special encoding for negative numbers and non-integer values.
- **Two's Complement Encoding**:
  - A method used in computing to encode negative numbers in binary.
  - It allows the representation of negative integers in a binary system, facilitating arithmetic operations involving negative numbers.
- **Floating-Point Notation**:
  - Used to represent real numbers (numbers with fractional parts) in a binary system.
  - Essential for scientific computations, graphics rendering, and any application requiring precision beyond whole numbers.

1.4 How a Program Works

- **Understanding the CPU and Machine Language**:
  - The CPU (Central Processing Unit) only understands machine language, composed of binary instructions.
  - Fundamental operations include reading data, arithmetic operations (addition, subtraction, multiplication, division), data movement, and comparison.
  - Machine language programming is highly challenging due to its complexity and susceptibility to errors.

1.4.1 From Machine Language to Assembly

- **The Role of Assembly Language**:
  - Serves as a more human-readable form of machine language using mnemonic codes instead of binary.
  - An assembler translates assembly language into machine language, making the development process more manageable.
  - Although more understandable than machine language, assembly language is still low-level and specific to processor architectures.

Python's Place in Programming Languages

- **High-Level Language**:
  - Python is a high-level programming language, meaning it is far removed from machine language and more understandable by humans.

- It abstracts the complexities of machine language, allowing developers to write programs without dealing with the intricacies of the CPU's operations.
- Python code is interpreted or compiled into machine language, making it executable on a computer.

Exercises and Further Learning

- **Practical Application**:
  - Write Python scripts to demonstrate two's complement and floating-point operations.
  - Explore how Python handles arithmetic operations at a low level.
  - Compare Python programming with basic assembly language tasks to understand the abstraction level.

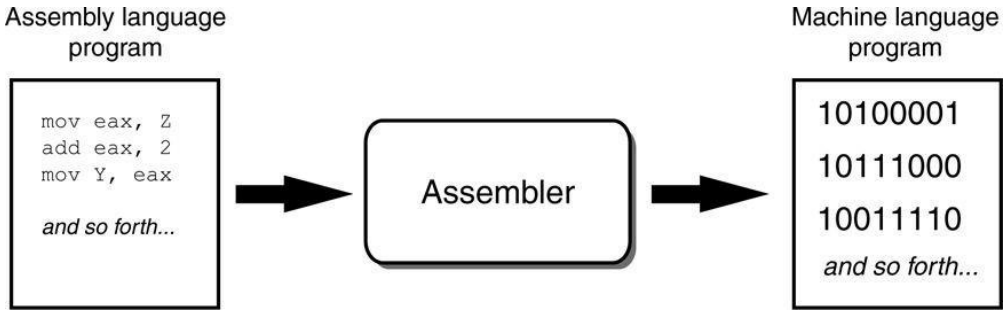


Figure 1-9 An assembler translates an assembly language program to a machine language program

1.4.2 High-Level Languages

Understanding High-Level Languages

High-level languages represent a significant step in the evolution of programming languages, offering a more abstracted way of coding compared to low-level languages like Assembly. These languages are characterized by their readability and simplicity, making programming more accessible and less dependent on the specifics of computer architecture.

Characteristics of High-Level Languages

- **Abstraction from Hardware**: Unlike low-level languages, high-level languages do not require a deep understanding of the computer's hardware or CPU architecture. This abstraction allows programmers to focus more on solving problems and developing algorithms without worrying about hardware-specific details.
- **Ease of Use**: High-level languages are often more straightforward and readable, using syntax and keywords that resemble human language or mathematical notation. This makes them more accessible to beginners and versatile for various applications.

Examples

- Common examples of high-level programming languages include Python, C++, Ruby, Ada, BASIC, and Visual Basic. Each of these languages has its unique features and areas of application, but they all share the trait of being more abstracted from machine-level instructions.

High-Level vs. Low-Level

- In comparison to assembly language, which requires detailed knowledge of the CPU and is closer to machine language, high-level languages simplify many tasks. A complex task in assembly might require several intricate instructions, while in a high-level language, the same task could be achieved with fewer and simpler commands.

Key Words, Operators, and Syntax: An Overview

Key Words

- **Reserved Words**: These are predefined words in a programming language that have special meanings. They are the building blocks of the language's syntax and are used to perform various programming functions.

Operators

- **Functionality**: Operators in high-level languages are used to perform operations on data. These include arithmetic operations, logical operations, comparison operations, and more.

Syntax

- **Rules**: The syntax of a language is a set of rules and guidelines that dictate how the code must be written and structured. Correct syntax is essential for the program to be understood and executed by the computer.

Statements

- **Instructions**: A statement in a programming language is an individual instruction that performs a specific operation. A program is essentially a collection of these statements that collectively perform a complex task.

1.4.3 Compilers and Interpreters

Source Code

- The code written in a high-level language is known as source code. It's the human-readable set of instructions that needs to be converted into machine language for the computer to execute.

Compilers and Interpreters

- **Compiler**: A compiler is a program that translates the entire source code into a machine language program (executable file) in one go. This process is done before the program is run.
- **Interpreter**: An interpreter, on the other hand, translates the source code into machine language on the fly, executing it line by line. This means the code is being translated and run simultaneously.

Syntax Errors

- Errors such as misspellings, incorrect punctuation, or improper use of operators are called syntax errors. These errors prevent the code from being correctly translated into machine language, leading to program failures or unexpected behavior.

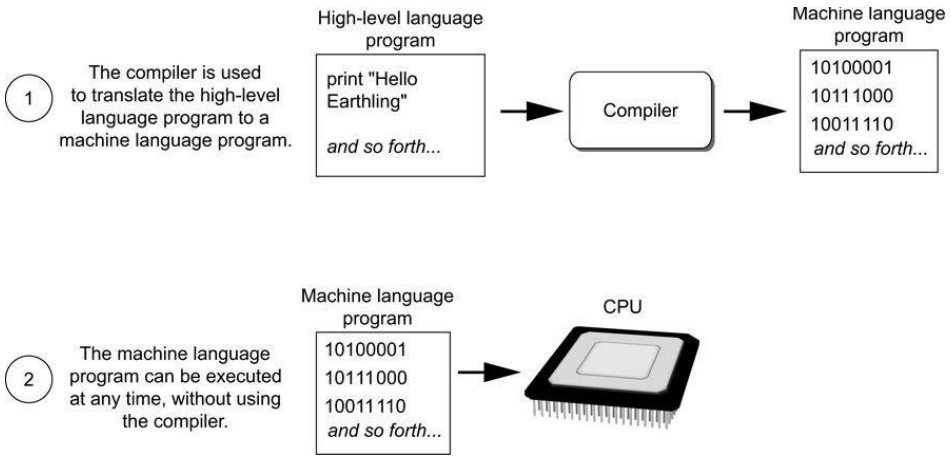


Figure 1-10 Compiling a high-level program and executing it

An interpreter is a program that both translates and executes the instructions in a high-level language program.

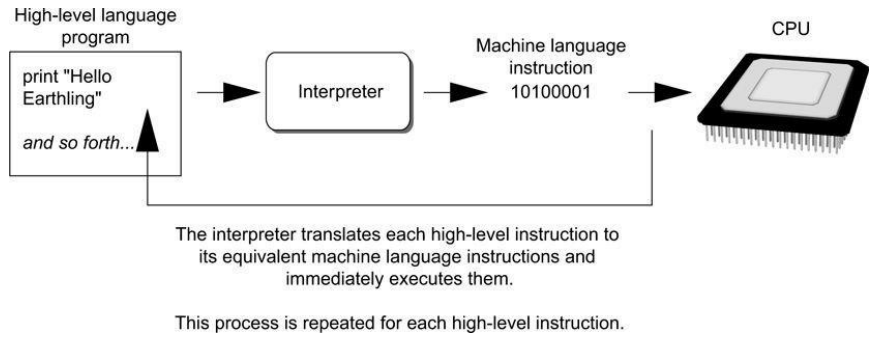


Figure 1-11 Executing a high-level program with an interpreter

1.5 Using Python

Introduction to the Python Interpreter

The Python interpreter is a core component of the Python programming environment. It serves as the engine that reads and executes Python code. When you install Python on your computer, the interpreter is one of the primary tools that get installed. Its role is to convert Python code into instructions that the computer can understand and execute.

Modes of the Python Interpreter

1. **Interactive Mode**:
    - **Description**: In this mode, the Python interpreter waits for Python commands entered through the keyboard.
    - **Execution**: Each statement is executed immediately after it is input, making it an excellent tool for experimenting with Python's features and for learning the language.
2. **Script Mode**:
    - **Description**: In script mode, the interpreter reads and executes Python code from a file, typically known as a Python script or program.
    - **Usage**: This mode is used for running complete Python programs written in **.py** files.
    - **Reference**: For more detailed information on using Python in script mode, refer to page 21 of the prescribed textbook.

1.5.1 The IDLE Programming Environment

Overview of IDLE

IDLE (Integrated DeveLopment Environment) is the standard programming environment provided with Python. It is designed to facilitate the writing, testing, and debugging of Python code.

Features of IDLE

1. **Automatic Installation**:
    - IDLE is automatically installed alongside Python, providing a ready-to-use environment for writing Python code.
2. **Built-in Text Editor**:
    - **Functionality**: IDLE comes with an integrated text editor that is used to write Python scripts.
    - **Features**: The editor provides basic functionalities like editing, saving, and running Python scripts.
3. **Code Colorization**:
    - **Purpose**: IDLE editor colorizes code to enhance readability and to make it easier to distinguish between different elements of the code, such as keywords, variables, strings, and comments.
    - **Benefits**: This feature helps in identifying syntax structures, making it easier for beginners to understand the structure of the Python language.

Additional Considerations

- IDLE is particularly well-suited for beginners due to its simplicity and ease of use. However, for more advanced programming or larger projects, other development environments or text editors may be preferred.

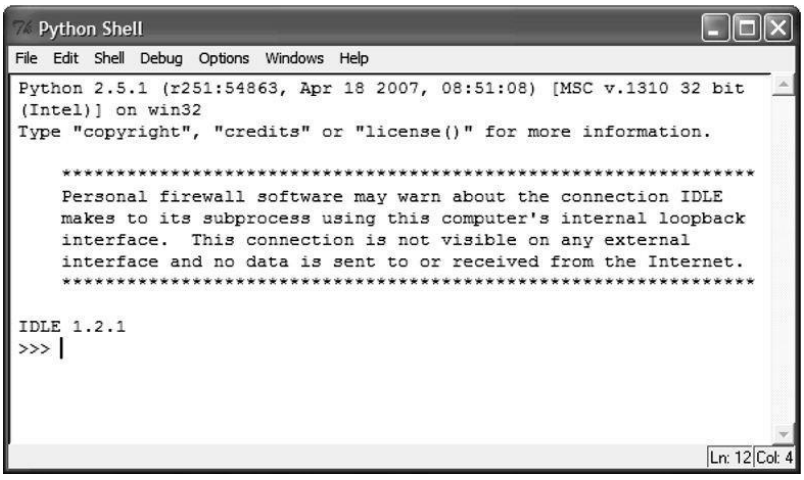


Figure 1-12 IDLE



Review Questions

A(n) _____ is a set of instructions that a computer follows to perform a task. a. compiler

b. program

c. interpreter

d. programming language

2. The physical devices that a computer is made of are referred to as _____. a. hardware

b. software

c. the operating system

d. tools

3. The part of a computer that runs programs is called _____.

a. RAM

b. secondary storage

c. main memory

e. the CPU

4. A component that collects data from people or other devices and sends it to the computer is called _____.

a. an output device

b. an input device

c. a secondary storage device

d. main memory

5. A video display is a(n) _____ device.

a. output

b. input

c. secondary storage

d. main memory

6. A _____ is enough memory to store a letter of the alphabet or a small number. a. byte

b. bit

c. switch

d. transistor

7.The words that make up a high-level programming language are called _____. a. binary instructions

b. mnemonics

c. commands

d. keywords

8. The rules that must be followed when writing a program are called _____. a. syntax

b. punctuation

c. keywords

d. operators

9. A(n) _____ program translates a high-level language program into a separate machine language program.

a. assembler

b. compiler

c. translator

d. utility

True/False

10. A syntax error does not prevent a program from being compiled and executed.
11. Windows, Linux, Android, iOS, and macOS are all examples of application software.
12. Word processing programs, spreadsheet programs, email programs, web browsers, and games are all examples of utility programs.

Exercise

To make sure that you can interact with the Python interpreter, try the following steps on your computer:

• Start the Python interpreter in interactive mode.

• At the >>> prompt, type the following statement then press Enter:

print('This is a test of the Python interpreter.') Enter

• After pressing the Enter key, the interpreter will execute the statement. If you typed everything correctly, your session should look like this:

>>> print('This is a test of the Python interpreter.') Enter

This is a test of the Python interpreter.

>>>

• If you see an error message, enter the statement again, and make sure you type it exactly as shown.

- Exit the Python interpreter. (In Windows, press Ctrl-Z followed by Enter. On other systems, press Ctrl-D.)

Refer to the Prescribed Textbook for more questions

TOPIC 2
PYTHON FUNDAMENTALS



Learning Outcomes

After studying this topic you should be able to:

- Design a program
- Utilize the print function for output display.
- Incorporate comments and manipulate variables.
- Read input from the keyboard.
- Execute calculations and perform string concatenation.
- Produce formatted output using F-strings.
- Implement named constants for improved code clarity.

2.1 Introduction

This topic delves into the essentials of program development, emphasizing variables, data types, and sequence structures. You'll learn to craft programs that interact with users, perform calculations, and output results in a structured format. Tools such as pseudocode and flowcharts are introduced to aid in program design.

2.2 Designing a Program

The Program Development Cycle (PDC)

Effective programming starts with a well-thought-out design. The PDC guides programmers through essential stages to ensure their programs are both functional and efficient.

Five Phases of the PDC:

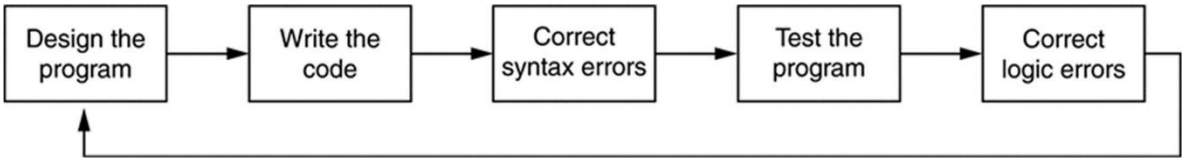
Design the Program: Planning the program's structure and functionality.

Write the Code: Translating the plan into executable code.

Correct Syntax Errors: Refining the code by fixing syntax-related issues.

Test the Program: Verifying the program works as intended.

Correct Logic Errors: Resolving errors in the program's logic.



Refer to Figure 2-1 on page 31 of the recommended textbook for a visual representation of the PDC.

Steps in Program Design:

1. **Understand the Task**: This involves comprehending the program's purpose, often derived from client requirements.
2. **Determine Steps to Perform the Task**: Developing an algorithm, which is a series of logical steps to achieve the task.

Example: Program to Calculate Gross Pay

- Obtain the number of hours worked.
- Determine the hourly pay rate.
- Multiply hours by the pay rate to calculate gross pay.
- Display the gross pay result.

Pseudocode

Pseudocode is an informal, non-executable way of outlining a program's logic. It helps programmers focus on the design without getting bogged down by syntax.

Example of Pseudocode:

- Input the hours worked.
- Input the hourly pay rate.
- Calculate gross pay (hours worked × hourly rate).
- Display the gross pay.

Flowcharts

Flowcharts offer a visual representation of a program's logic, making them invaluable for understanding and conveying complex processes.

Common Flowchart Symbols:

- **Rectangle**: A process or operation.

- **Diamond**: A decision point in the program.
- **Arrow**: The direction of process flow.
- **Terminal**: Marks the start or end of the flowchart.
- **Process**: An action or operation.
- **Decision**: A point where the flow diverges based on a condition.
- **Connector**: Links different parts of the flowchart.

Flowcharts simplify logic, highlight potential errors, and enhance the structure and clarity of code. They are essential for ensuring that code is organized, comprehensible, and maintainable.

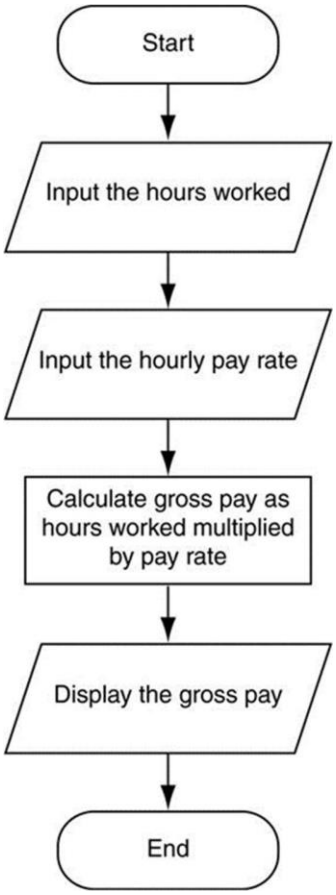


Figure 2-2 Example flowchart for the pay calculating program

The arrows that connect the symbols represent the "flow" of the program. You start at the Start terminal and follow the arrows to the End terminal in order to step through the symbols in the correct order.

2.3 Input, Processing, and Output

Computer programs typically perform these three crucial steps:

1. **Input Reception**: Gathering data from external sources like keyboard input.
2. **Processing**: Performing operations on the received input.
3. **Output Generation**: Producing results based on the processed data.

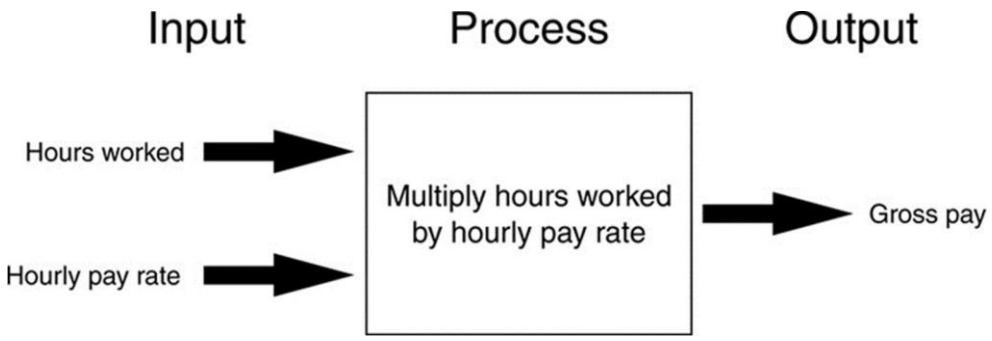


Figure 2-3 The input, processing, and output of the pay calculating program

2.4 Displaying Output with the print Statement

- **print Function**: In Python, **print** is a built-in function used for displaying output.
- **Example**: To display "Python Programming", use **print("Python Programming")**. The argument passed within the parentheses is the data to be displayed on the screen.

Outputting Names of African Countries:

```
print("South Africa")
print("Zimbabwe")
print("Zambia")
print("Botswana")
print("Malawi")
```

Output:

South Africa

Zimbabwe

Zambia

Botswana

Malawi

2.4.1 Strings and String Literals

- **String Literals**: Strings in Python are sequences of characters used as data. When a string appears directly in code, it's called a string literal.
- **Usage**: Strings can be enclosed in single, double, or triple quotes.
- **Single and Double Quotes**: Use double quotes if the string contains a single quote (apostrophe) and vice versa.

Examples:

```
print("Don't hate your neighbour!")
print('Your assignment is to read "Hamlet" by tomorrow.')
```

Output:

Don't hate your neighbour!

Your assignment is to read "Hamlet" by tomorrow.

- **Triple Quotes**: Used for multi-line strings, preserving formatting and whitespace.

Example with Triple Quotes:

```
string_1 = '''Hello,
World!'''
print(string_1)
```

Output:

Hello,
World!

2.5 Comments

- **Purpose**: Comments are non-executable statements used to explain and annotate code.
- **Types**:
  - **Single-line Comments**: Start with # and extend to the end of the line.
  - **Multi-line Comments**: Enclosed in triple quotes (''' or """).

Examples:

```
# This is a single-line comment """ This is a
multi-line comment """
```

2.6 Variables

- **Definition**: Variables in Python are named locations in memory for storing data values.
- **Assignment**: Use the assignment operator = to assign values to variables.

Examples:

```
x = 42
y = "Hello, World!"
z = [1, 2, 3]
```

- **Type Inference**: Python automatically determines the type of a variable based on the assigned value.

Variable Naming Best Practices:

- **Naming Rules**:
  - Must begin with a letter or underscore (_).
  - Can include letters, digits, or underscores after the first character.
  - Should not be a reserved word in Python.
  - Case-sensitive.
  - Cannot contain spaces.
- **Readability**: Use lowercase with underscores for readability (e.g., **age_of_student**).

2.6.1 Variable Naming Rules

When naming variables in Python, follow these guidelines for clarity and to adhere to Python's syntax rules.

Example:

```
age = 25
```

2.6.1 Variable Naming Rules

Mandatory Rules:

1. **Start Character**: A variable name must begin with a letter (either uppercase or lowercase) or an underscore (_). It cannot start with a digit.
2. **Subsequent Characters**: Following the initial letter or underscore, the variable name can include:
    - Letters (a-z, A-Z)
    - Digits (0-9)
    - Underscores (_)
3. **Reserved Words**: Variable names cannot be the same as Python's reserved words (keywords) like **for**, **while**, **if**, etc. These words have special meanings in Python and using them as variable names will lead to syntax errors.
4. **Case Sensitivity**: Variable names are case-sensitive in Python. This means **age**, **Age**, and **AGE** are treated as three distinct variables.
5. **No Spaces**: Spaces are not allowed within variable names. They must be a single continuous string of characters.

Conventions for Enhanced Readability:

1. **Lowercase with Underscores**: It is a common practice to use lowercase letters with underscores separating words for variable names. This format enhances readability, especially for longer names. For example, **age_of_student**, **gross_pay**, **hot_dogs_sold_today**.
2. **CamelCase Convention**: Alternatively, the CamelCase naming convention can be used, particularly in contexts where it aligns with the surrounding code's style (e.g., certain class names). In CamelCase, each word or abbreviation in the middle of the phrase begins with a capital letter, with no intervening spaces or punctuation. Examples include **grossPay**, **hotDogsSoldToday**.

Examples:

- Valid Variable Names:
    - **username**
    - **inventory_list**
    - **_tempValue**
    - **numberOfUsers**
- Invalid Variable Names:

- **2ndAttempt** (starts with a digit)
- **user-name** (contains a hyphen)
- **global** (reserved word)
- **full name** (contains a space)

Note:

While Python allows the use of both lowercase with underscores and CamelCase, it's crucial to maintain consistency within your project. Adhering to one style throughout a project aids in readability and understanding of the code by other developers.

Sample variable names

Variable Name	Legal or Illegal?
`units_per_day`	Legal
`dayOfWeek`	Legal
`3dGraph`	Illegal. Variable names cannot begin with a digit.
`June1997`	Legal
`Mixture#3`	Illegal. Variable names may only use letters, digits, or underscores.

2.6.2 Displaying Multiple Items with the print Statement

Basic Usage:

- In Python, the **print()** function is used to output data to the standard output device (screen).
- You can display multiple items by separating them with commas.

Examples:

1. **Basic Multiple Printing:**

```
print("First item:", 1)
print("Second item:", 2)
# Output: # First item: 1 # Second item: 2
```

2. **Displaying Variables:**

```
# This program demonstrates variable printing.
room = 503 print('I am staying in room number', room)
# Output: # I am staying in room number 503
```

Using end Parameter:

- The **end** parameter in the **print()** function specifies what to print at the end of the output.
- By default, **end** is set to **'\n'** (newline character), but you can change it to space, comma, or any other string.

```
print("First item:", 1, end=", ")
 print("Second item:", 2) # Output: # First item: 1, Second item: 2
```

Using sep Parameter:

- The **sep** parameter defines the separator between multiple items.
- By default, **sep** is a space, but it can be modified to other characters.

```
print("First item:", 1, sep=" | ")
print("Second item:", 2, sep=" | ")
```

Output: # First item: | 1 # Second item: | 2

Variable Reassignment

- In Python, variables can be reassigned to new values using the assignment operator (=).
- Once a variable is assigned a new value, it refers to this value until it is reassigned again.

Example:

x = 10

print(x) # Output: 10

x = 20 print(x) # Output: 20

- Initially, **x** is assigned the value 10. Later, **x** is reassigned to 20. The second **print()** statement reflects this change.

Here is another example:

```
1  # This program demonstrates variable reassignment.
2  # Assign a value to the dollars variable.
3  dollars = 2.75
4  print 'I have', dollars, 'in my account.'
5
6  # Reassign dollars so it references
7  # a different value.
8  dollars = 99.95
9  print 'But now I have', dollars, 'in my account!'
```

Variable Assignment and Reassignment

Creating and Using Variables:

1. **Assignment of Variable:**

In Python, when you create a variable and assign it a value, the variable acts as a reference to that value in memory. For example:

dollars = 2.75 # Line 3: Variable 'dollars' is created and assigned the value 2.75

2. **Reassignment of Variable:**

You can change the value of a variable at any point in your program. This is known as reassignment. For example:

dollars = 99.95 # Line 8: Variable 'dollars' is given a new value 99.95

After this reassignment, the variable **dollars** now refers to the new value **99.95**.

Memory Management and Garbage Collection

- **Previous Value in Memory:**

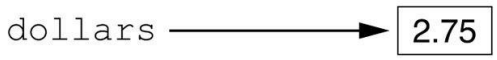
When a variable is reassigned, the old value (**2.75** in our example) remains in the computer's memory temporarily. However, since it's no longer referenced by the **dollars** variable, it becomes inaccessible through your program.

- **Garbage Collection:**

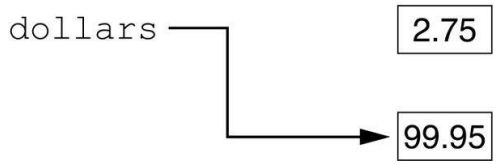
Python uses a mechanism called "garbage collection" to reclaim memory occupied by unreferenced values. This process automatically and periodically looks for objects in memory that are no longer in use (i.e., not referenced by any variable) and frees up the memory they occupy.

- **Automatic Memory Management:** This aspect of Python's design simplifies memory management for developers. Unlike languages where memory management is manual, Python automates this process, reducing the risk of memory leaks.
- **Efficiency and Optimization:** While the Python garbage collector is efficient, it is still advisable for programmers to manage resources wisely, especially in large-scale applications where resource management is crucial for performance.

The dollars variable after line 3 executes.



The dollars variable after line 8 executes.



2.6.3 Numeric Data Types and Literals

Understanding Numeric Literals:

- **Numeric Literal**: In programming, a numeric literal is a direct representation of a number in the code. In Python, numeric literals are categorized into different data types based on their format and the kind of number they represent.

Types of Numeric Literals:

1. **Integer (int):**
    - An integer is a numeric literal without a decimal point.
    - It can be positive, negative, or zero.
    - Examples: **7124**, **503**, **-9**.
2. **Floating-Point Number (float):**
    - A float is a numeric literal with a decimal point.
    - It represents real numbers and can include fractions.
    - Examples: **1.5**, **3.1415**, **5.0**.

Determining Data Types with type() Function:

- Python provides a built-in function **type()** to determine the data type of a value.
- The output of the **type()** function is in the format **<class 'type'>**, indicating the data type of the provided argument.

Examples:

print(type(1)) # Output: <class 'int'> print(type(1.0)) # Output: <class 'float'>

- In these examples, **1** is identified as an integer (**int**), and **1.0** as a floating-point number (**float**).

2.6.4 Storing Strings with the str Data Type

The String Data Type (str):

- In Python, strings are sequences of characters used to store textual data.
- The **str** data type represents strings.
- Strings in Python are enclosed in quotes, which can be single (**'...'**), double (**"..."**), or triple-quoted (**'''...'''** or **"""..."""** for multi-line strings).

Creating and Using Strings:

greeting = "Hello, world!" name = 'John Doe' multiline_string = """This is a multiline string."""

- Here, **greeting** and **name** are simple one-line strings, while **multiline_string** spans multiple lines.

String Operations:

- Strings in Python are objects and come with various built-in methods for manipulation, such as **upper()**, **lower()**, **strip()**, **split()**, etc.
- Strings are immutable, meaning once they are created, their contents cannot be changed.

```
1  # Create variables to reference two strings.
2  first_name = 'Kathryn'
3  last_name = 'Marino'
4
5  # Display the values referenced by the variables.
6  print(first_name, last_name)
```

Source: Gaddis(2021) page 46.

Reassigning a Variable to a Different Type

Dynamic Typing in Python:

- In Python, variables are dynamically typed. This means that a variable's data type is not explicitly declared and can change during the execution of a program.

Example and Explanation:

Consider the following example:

string1 = 'Hello World'

print(type(string1)) # Output: <class 'str'>

string1 = 123

print(type(string1)) # Output: <class 'int'>

In this example, string1 is first assigned to a string 'Hello World'. Its type is <class 'str'>. Later, string1 is reassigned to an integer 123, changing its type to <class 'int'>.

Caution in Dynamic Typing:

- While dynamic typing offers flexibility, it requires careful management to avoid errors.
- Mismanagement can lead to unexpected behavior or runtime errors, especially in complex programs.



Think Point:

Consider the following assignments:

value1 = 99

value2 = 45.9

value3 = 7.0

value4 = 7

value5 = 'abc'

After execution, the Python data types are:

value1: <class 'int'>

value2: <class 'float'>

value3: <class 'float'>

value4: <class 'int'>

value5: <class 'str'>

2.6.4 Reading Input from the Keyboard

Using the input() Function:

The input() function in Python reads a line of text from the user and stores it as a string.

Example:

Reading and using user input:

name = input('What is your name? ')

print('Hello, ' + name)

This code will prompt the user with "What is your name?" and store the user's response in the name variable.

Handling Numeric Input:

The value returned by input() is always a string. To use it as a number, you must explicitly convert it.

Example with Numeric Conversion:

Converting user input to an integer:

age = input('What is your age? ')

age = int(age)

print('You are ' + str(age) + ' years old.')



Here, the user's age is initially read as a string, then converted to an integer using int(age). The print() function outputs the age, converting it back to a string with str(age).Here, the user's age is initially read as a string, then converted to an integer using int(age). The print() function outputs the age, converting it back to a string with str(age)

```
1  # Get the user's name, age, and income.
2  name = input('What is your name? ')
3  age = int(input('What is your age? '))
4  income = float(input('What is your income? '))
5
6  # Display the data.
7  print('Here is the data you entered:')
8  print('Name:', name)
9  print('Age:', age)
10 print('Income:', income)
```

Source: Gaddis (2021) page 52

Output

Program Output (with input shown in bold)

What is your name? Chris Enter

What is your age? 25 Enter

What is your income? 75000.0 Here is

the data you entered:

Name: Chris

Age: 25

Income: 75000.0

2.7 Performing Calculations

In Python, the following are the basic mathematical operators:

Symbol	Operation	Description
+	Addition	Adds two numbers
–	Subtraction	Subtracts one number from another
*	Multiplication	Multiplies one number by another
/	Division	Divides one number by another and gives the result as a floating-point number
//	Integer division	Divides one number by another and gives the result as a whole number
%	Remainder	Divides one number by another and gives the remainder
**	Exponent	Raises a number to a power

Example a =

5 b =

3

addition = a + b subtraction = a - b

multiplication = a * b division = a / b

floor_division = a // b modulus = a %

b exponentiation = a ** b

print("Addition: ", addition) # Output: Addition: 8 print("Subtraction: ",

subtraction) # Output: Subtraction: 2 print("Multiplication: ",

multiplication) # Output: Multiplication: 15 print("Division: ", division)

Output: Division:

1.6666666666666667

print("Floor Division: ", floor_division) # Output: Floor Division: 1

```
print("Modulus: ", modulus)          # Output: Modulus: 2 print("Exponentiation:    ",
    exponentiation)          #          Output: Exponentiation: 125
```

Integer Division

Integer division is the behaviour that results when an integer is divided by another integer resulting in the answer being an integer as well.

For Example:

num_add = 17 + 5 # the answer will be 22 num_div_int = 3 / 2

the answer is 1; 0.5 is the fractional part and it is truncated.

num_div_real = 3.0 / 2.0 # the answer is 1.5

2.7.1 Operator Precedence

In Python, the order of operations (precedence) for mathematical operators is determined by the following rules:

1. Parentheses (()) first
2. Exponentiation (**) second
3. Multiplication (*), division (/), and floor division (//) are next, evaluated from left to right.
4. Addition (+) and subtraction (-) last, evaluated from left to right.

Here is an example to demonstrate the operator precedence in Python:

```
    a = 5 b =
    3 c = 2
result = a + b * c ** 2 print(result)  #
    Output: 29
```

In this example, c is first raised to the power of 2, then multiplied by b, and finally the sum of a and the previous result is obtained.

Example Expressions

Expression	Value
`5 + 2 * 4`	13
`10 / 2 – 3`	2
`8 + 12 * 2 – 4`	28
`6 – 3 * 2 + 7 – 1`	6

Grouping with Parentheses

Expression	Value
`(5 + 2) * 4`	28
`10 / (5 – 3)`	5
`8 + 12 * (6 – 2)`	56
`(6 – 3) * (2 + 7) / 3`	9

Source: Gaddis (2021) page 58

2.7.2 The Exponent and Remainder Operators

** is the exponent operator. Its purpose is to raise a number to a power.

For Example: a = 2 b = 3 c = a **

b print(c) # Output: 8

% is the remainder operator. Its purpose is to perform division and return the remainder.

For Example: leftover = 17 % 3 #remainder is 2

Converting math formulae to programming statements

Algebraic Expression

$$x = \frac{a + b}{3}$$

Programming Expression

```
x = (a + b) / 3
```

Algebraic Expression	Python Statement
$y = 3\frac{x}{2}$	`y = 3 * x / 2`
$z = 3bc + 4$	`z = 3 * b * c + 4`
$a = \frac{x + 2}{b - 1}$	`a = (x + 2) / (b - 1)`

2.7.3 Data Type Conversion

Python follows the following rules when evaluating mathematical expressions:

- When an operation is performed on two int values, the result will be an int.
- When an operation is performed on two float values, the result will be a float.
- When an operation is performed on an int and a float, the int value will be temporarily converted to a float and the result of the operation will be a float.

float() and int() are two of python's built in functions.

int(item): You pass an argument to the int() function and it returns the argument's value converted to an int.

float(item): You pass an argument to the float() function and it returns the argument's value converted to a float.

For example:

```
x  = 27.9
y  = int(x)              # y will be assigned 27 x = -12.9
y = int(x)      # y will be assigned -12 y = 7
   x = float(y)                    # x will be assigned 7.0
```

In Python, statements can be broken down into multiple lines by using the line continuation character (\ ).

For example:

```
units_sold = 10 sales_amount = 100
print('We sold', units_sold, 'for a total of', sales_amount)
var1 = 1 var2 = 2
var3 = 3 var4 = 4
result = var1 * 2 + var2 * 3 + \         var3 * 4 +
var4 * 5
print(result)
```

2.8 Escape Characters and String Concatenation

In Python, escape characters are used to represent certain special characters within a string. The backslash (\) is used as an escape character to indicate that the character following it should be treated specially.

A special character that is preceded with a backslash (\), appearing inside a literal.

Escape Character	Effect
\n	Causes output to be advanced to the next line.
\t	Causes output to skip over to the next horizontal tab position.
\'	Causes a single quote mark to be printed.
\"	Causes a double quote mark to be printed.
\\	Causes a backslash character to be printed.

For example:

```
print ‘One\nTwo\nThree’
```

Output

One Two

Three

String concatenation is achieved through the use of the string concatenation operator (+).

For example:

```
print('this is' + 'one string.') # one string is appended to another Output
This is one string
```

In Python, Implicit String Literal Concatenation is a feature that allows multiple string literals to be concatenated together without the need for explicit concatenation operators such as + or \ For example:

```
print("Hello "       "world")
```

Output

Hello world

This feature can be useful for breaking up long string literals into multiple lines, making the code easier to read and maintain. Note that this only works for string literals and not for variables or expressions.

More arguments for the print Function

In Python, you can use multiple print statements to print on the same line by using the end parameter of the print function. By default, end is set to '\n', which means a new line will be added after each print statement. You can change the value of end to “ “ (an empty string) to prevent the addition of a new line.

Here's an example:

```
print("This is the first part", end="") print(" and this is the
second part.")
```

Output

This is the first part and this is the second part.

You can also specify an item separator for print statements by using the sep parameter of the print function. By default, sep is set to ' ' (a single space), but you can change it to any string value you want.

For example:

```
print(1, 2, 3, sep=", ")
```

Output

1, 2, 3

2.9 Formatting

2.9.1 Formatting the output of a print statement

You can format the output of a print statement in a variety of ways. Some of the most common methods include:

1. Concatenation: You can concatenate strings using the + operator, and you can convert non-string values to strings using the str function. name = "John" age = 32

   ```
   print("My name is " + name + " and I am " + str(age) + " years old.")
   ```

2. format() function: To format the output of a print statement in Python, you can use the format() method. This method allows you to specify placeholders in the string to be printed, and then replace those placeholders with actual values. For example: name = "John Doe" age = 30

   ```
   print("My name is {} and I am {} years old.".format(name, age))
   ```

   Output

My name is John Doe and I am 30 years old.

3. Using f-strings: You can also use f-strings, which are a more recent way to format strings in Python, introduced in Python 3.6: name = "John Doe" age = 30

print(f "My name is {name} and I am {age} years old.")

Output

My name is John Doe and I am 30 years old.

This would give the same output as the format() method.

2.9.2 Formatting Numbers

Float-point Numbers, without formatting the number can be displayed with up to 12 significant digits. The % symbol is a string format operator when the operand on the left side of the % is a string. A formatting specifier is a special set of characters that specify how a value should be formatted.

For example:

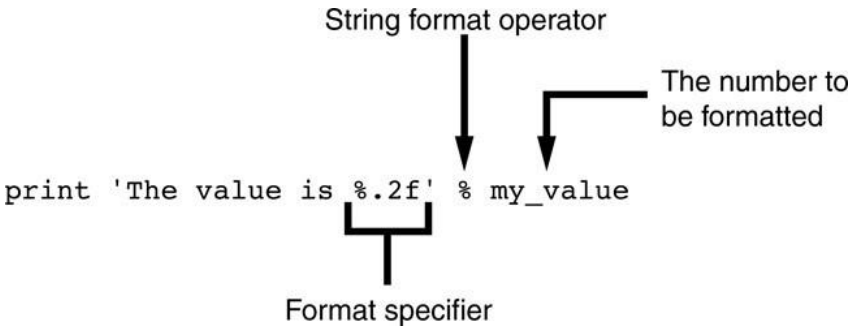
my_value = 7.23456

print ("The value is %.2f" % my_value)

Output

The value is 7.23

In the above code, %.2f is a placeholder for the formatted floating-point number my_value with 2 decimal places. The % operator replaces the placeholder with the actual value, and the resulting string is printed to the console.



Here, .2f specifies that the number should be formatted as a floating point with 2 decimal places.

Another example:

```
1  # This program demonstrates how a floating-point
2  # number can be formatted.
3  amount_due = 5000.0
4  monthly_payment = amount_due / 12.0
5  print 'The monthly payment is %.2f' % monthly_payment
```

Formatting Multiple Values For

Example:

value1 = 6.7891234 value2 =

1.2345678

print ‘The values are %.1f and %.3f’ % (value1, value2)

Output

The values are 6.8 and 1.235

You can also format integers and other numerical values in a similar way. For example:

integer = 42

print(f"The integer is: { integer:d }")

Output

The integer is: 42

Here, d specifies that the value should be formatted as a decimal integer.

Specifying a Minimum Field Width

The %d formatting specifier is used for formatting an integer while the %s specifier is used to format a string.

For example:

name = ’Ringo’

day = ‘Monday’

sales = 8450.5543

print (‘Hello %s. Good to see you!’ % name) print (‘The sales on %s were $%.2f.’ % (day, sales))

Output

Hello Ringo. Good to see you!

The sales on Monday were $8450.55.

2.10 Named Constants

Named constants are values that don't change throughout the execution of a program. While Python does not have a direct mechanism for declaring named constants, a common convention is to use all capital letters with underscores to indicate that the variable should not be changed.

For example:

PI = 3.14159265358979323846

GRAVITY = 9.8



This is just a convention and not enforced by the language itself, so it is still possible to change the value of these variables. To enforce the immutability of a value, one could use the constant library in Python or simply use a tuple or a namedtuple.

Here's an example of using a named constant: PI = 3.14159265 radius = 5 area = PI *

(radius**2) print("The area of a circle with radius", radius, "is", area)

Output

The area of a circle with radius 5 is 78.53975

In this example, the named constant PI is used in the calculation of the area of a circle. The value of PI is not expected to change, so it is created as a named constant. The radius is given a value of 5 and the area of the circle is calculated using the formula PI * (radius**2). The result is then printed to the console.



Think point: Despite named constants being mutable, why is it important to use them instead of magical numbers?



Review Questions

1.A _____ error does not prevent the program from running, but causes it to produce incorrect results.

a. syntax

b. hardware

c. logic

d. fatal

2. A _____ is a single function that the program must perform in order to satisfy the customer.

a. task

b. software requirement

c. prerequisite

d. predicate

3. An informal language that has no syntax rules and is not meant to be compiled or executed is called _____.

a. faux code

b. pseudocode

c. Python

d. a flowchart

4. A _____ is a diagram that graphically depicts the steps that take place in a program.

a. flowchart

b. step chart

c. code graph

d. program graph

5. A _____ is a sequence of characters.

a. char sequence

b. character collection

c. string

d. text block

6. This symbol marks the beginning of a comment in Python. a. &

b. *

c. **

d. #

7. Which of the following statements will cause an error?

a. x = 17

b. 17 = x

c. x = 99999

d. x = '17'

8. Which built-in function can be used to convert an int value to a float? a. int_to_float()

b. float()

c. convert()

d. int()

9. What is the difference between floating-point division and integer division?

10. What is a magic number? Why are magic numbers problematic?

11. Complete the following table by writing the value of each expression in the Value column:

Expression	Value
6 + 3 * 5	_____
12 / 2 − 4	_____
9 + 14 * 2 − 6	_____
(6 + 2) * 3	_____
14 / (11 − 4)	_____
9 + 12 * (8 − 3)	_____

12. What value will be assigned to result after the following statement executes? result = 9 // 2

13. Write assignment statements that perform the following operations with the variables a, b, and c:

a. Adds 2 to a and assigns the result to b

b. Multiplies b times 4 and assigns the result to a

c. Divides a by 3.14 and assigns the result to b

d. Subtracts 8 from b and assigns the result to a

14. One acre of land is equivalent to 43,560 square feet. Write a program that asks the user to enter the total square feet in a tract of land and calculates the number of acres in the tract. Hint: Divide the amount entered by 43,560 to get the number of acres.

15. Assuming there are no accidents or delays, the distance that a car travels down the interstate can be calculated with the following formula:

Distance = Speed X Time

A car is traveling at 70 miles per hour. Write a program that displays the following:

- The distance the car will travel in 6 hours
- The distance the car will travel in 10 hours
- The distance the car will travel in 15 hours

TOPIC 3
DECISION STRUCTURES AND BOOLEAN LOGIC
Learning Outcomes



After studying this topic you should be able to:

- Write programs that use the if Statement
- Write programs that use the if-else Statement
- Write programs that compare strings
- Use nested decision structures and if-elif-else statement in writing programs
- Implement programs that use logical operators
- Implement programs that use boolean variables

3.1 Introduction

Decision can be made using decision structures and conditions. The student will gain knowledge of relational operators, Boolean expressions, and decision structures in this topic. They will also learn how to control the flow of a program. We examine the if, ifelse, and if-elif-else statements; also covered are nested decision structures and logical operators.

3.2 The if Statement

A control structure is a logical design that controls the order in which a set of statements execute.

A sequence structure is a set of statements that execute in the order that they appear.

```
print("Starting sequence") x = 2 + 3
print("The value of x is:", x)
```

The three statements above are executed in the order in which they appear.

On the other hand, a decision (or selection) structure is a control structure that can execute a set of statements and perform a specific action only if a certain condition exists. The most common way to implement a decision structure is with an if statement.

Decision (selection) structure: Diamond symbol represents a true/false condition

Single alternative decision structure provides only one alternative path of execution.

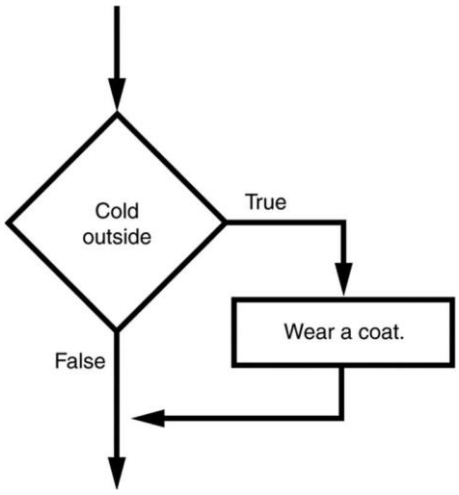


Figure 3.1 A simple decision structure

Decision (selection) structure:

In Python … if

condition:

statement

statement

statement etc.

In the above syntax, the 3 statements are executed only if the condition is true. x = 10

```
if x > 5:
    print("x is greater than 5")
```

In this example, the condition x > 5 is True, so the code inside the if statement will be executed and the output will be "x is greater than 5".

3.2.1 Boolean Expressions and Relational Operators

A relational operator determines whether a specific relationship exists between two values.

Relational operators are used to compare values and determine the relationship between them. The relational operators in Python are:

== (equal to)

!= (not equal to)

> (greater than)

< (less than)

>= (greater than or equal to)

<= (less than or equal to)

These operators return a boolean value of True or False based on the comparison between the values. For example:

3 == 3 This expression returns True because the two values being compared (3 and 3) are equal to each other.

"hello" == "hello" This expression returns True because the two strings being compared are identical.

3 != 4 This expression returns True because the two values being compared (3 and 4) are not equal to each other.

"hello" != "world" This expression returns True because the two strings being compared are not the same.

4 > 3 This expression returns True because 4 is greater than 3.

4 > 4 This expression returns False because 4 is not greater than 4.

3 < 4 This expression returns True because 3 is less than 4.

3 < 3 This expression returns False because 3 is not less than 3.

4 >= 4 This expression returns True because 4 is equal to 4, and 4 is also greater than or equal to 4.

1 <= 4 This expression returns True because 3 is less than or equal to 4.

Here is an example: # Three test scores test_score1 = 80
test_score2 = 90
test_score3 = 95

Calculate the average of the test scores average = (test_score1 + test_score2 + test_score3) / 3

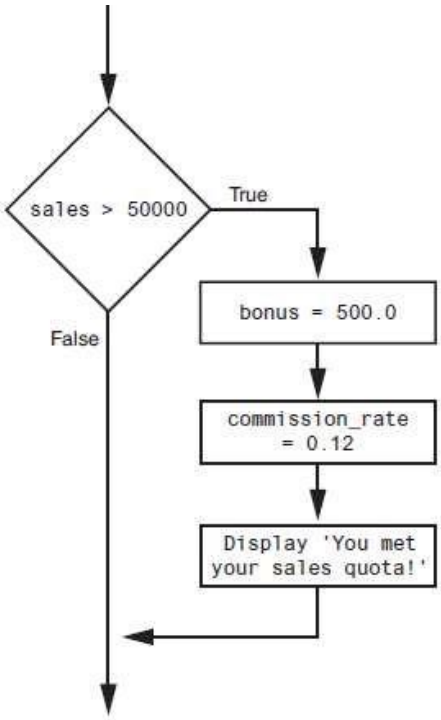
Use an if statement to check the value of the average if average > 90:
Display a message if the condition is True print("Congratulations! Your average score is above 90.")

Output
Congratulations! Your average score is above 90.

In this example, the average of the three test scores is calculated and stored in the variable average. Then, an if statement is used to check if the value of average is greater than 90. If the condition is true (i.e., average > 90), the message "Congratulations! Your average score is above 90." is displayed.

Here is another example that conditionally executes a block containing three statements:

```
if sales > 50000:
    bonus = 500.0 commission_rate = 0.12
    print('You met your sales quota!')
```



Source: Gaddis (2021) Page 125

3.3 The if-else Statement

A dual alternative decision structure provides two possible paths of execution – one path is taken if a condition is true, and the other path is taken if the condition is false.

In Python …

if condition:

statement

statement etc.

else:

statement statement

etc.

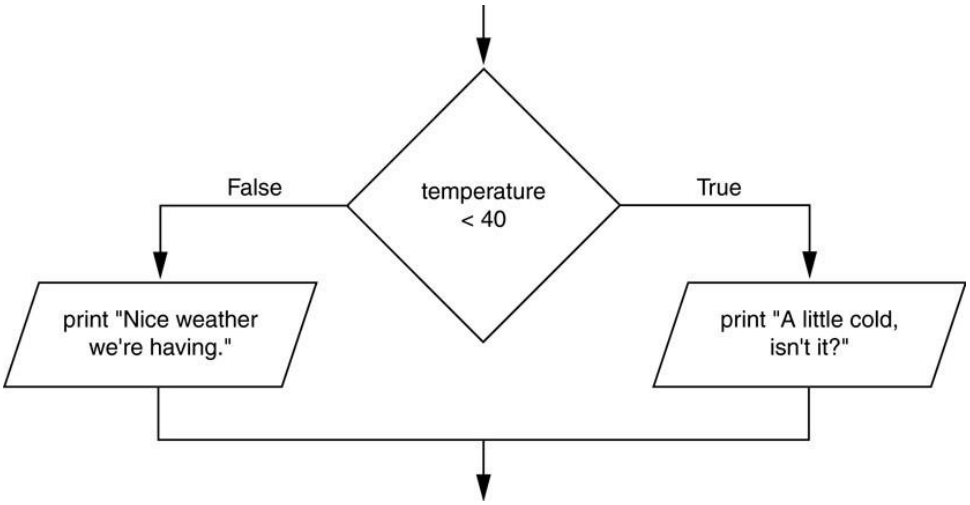


Figure 3-2 A dual alternative decision structure

```
if temperature < 40:
    print("A little cold, isn't it?")
else:
    print("Nice weather we're having.")
```

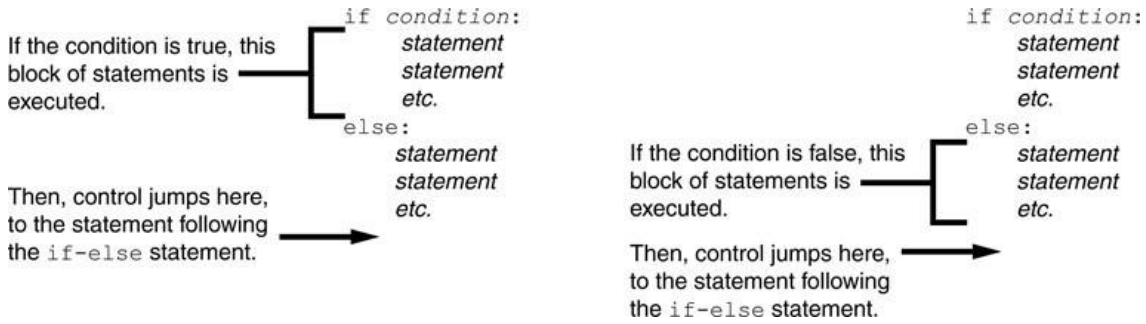


Figure 3-3 Conditional execution in an if-else statement

Source: Gaddis (2021) Page 125



Indentation is used to indicate blocks of code and determine the scope of variables and control structures. In an if-else statement, the code inside each block is indented from the if or else keyword. The standard indentation is 4 spaces, but you can use any number of spaces as long as you are consistent.

3.4 Comparing Strings

You can compare strings using the equal to (==) and not equal to (!=) operators, as well as the comparison functions such as "cmp()" (which is deprecated in Python 3).

In Python, string comparison is case-sensitive by default.

Here is an example:

```
# Define two strings string1 =
"Hello" string2 = "World"

# Compare the two strings using the equality operator if string1 == string2:
   print("The strings are equal.") else:
   print("The strings are not equal.")
```

Output

The strings are not equal.

To perform a case-insensitive comparison, you can convert the strings to either upper- or lower-case using the upper() or lower() methods and then compare them.

3.4.1 Other String Comparisons

In addition to determining whether strings are equal or not equal, you can compare two strings to determine if one is greater than or less than the other lexicographically (i.e. based on the dictionary order of their characters). This can be done using the greater than (>) and less than (<) operators. Here is an example:

```
string1 = "Hello" string2 =
"World"

# Compare the two strings using the greater than operator if string1 > string2:
   print("string1 is greater than string2.")
else:
   print("string1 is not greater than string2.")
```

Output

string1 is not greater than string2.

Here is another example:

```
# Compare the two strings using the less than operator if string1 < string2:
   print("string1 is less than string2.") else:
   print("string1 is not less than string2.")
```

Output

string1 is less than string2.

Note the following points about ASCII codes for representing characters:

- The uppercase characters A through Z are represented by the numbers 65 through 90.
- The lowercase characters a through z are represented by the numbers 97 through 122.
- When the digits 0 through 9 are stored in memory as characters, they are represented
- by the numbers 48 through 57. (For example, the string 'abc123' would be stored
- in memory as the codes 97, 98, 99, 49, 50, and 51.)
- A blank space is represented by the number 32.

When a program compares characters, it actually compares the codes for the characters.

3.5 Nested Decision Structures and the if-elif-else Statement

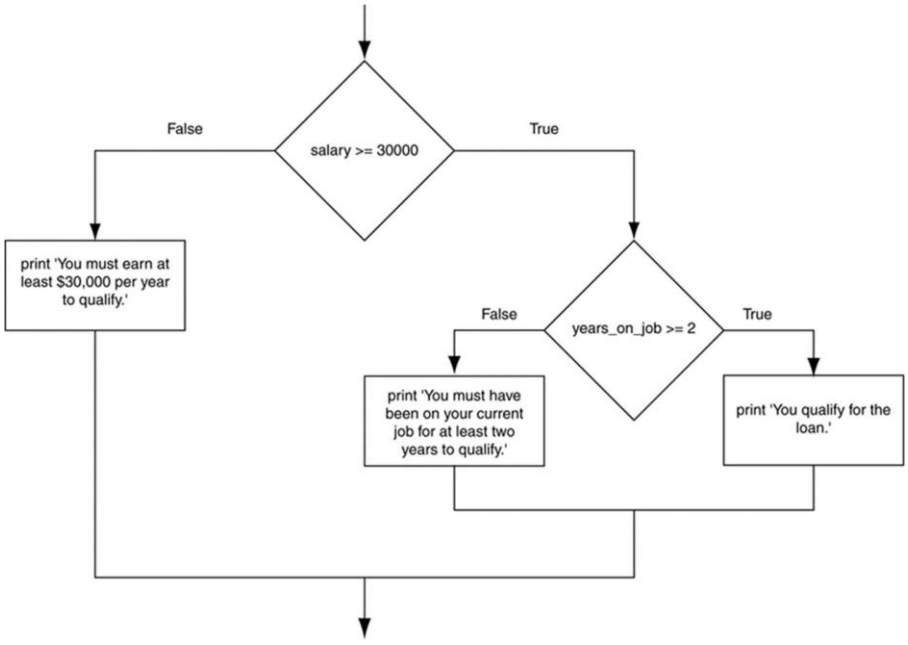


Figure 3-4 A nested decision structure

```
# This program determines whether a bank customer  # qualifies for
a loan.

 MIN_SALARY = 30000.0 # The minimum annual salary
 MIN_YEARS = 2 # The minimum years on the job

 # Get the customer's annual salary.  salary = float(input('Enter your
annual salary: '))

 # Get the number of years on the current job. years_on_job = int(input('Enter the
number of years employed: '))

 # Determine whether the customer qualifies.  if salary >=
MIN_SALARY:
                if years_on_job >= MIN_YEARS:
                        print('You qualify for the loan.')
```

```
            else:
                print(f'You must have been employed '
    f'for at least {MIN_YEARS} '                f'years to qualify.')
else:
            print(f'You must earn at least $'  f'{MIN_SALARY:,.2f} '
                    f'per year to qualify.')
```

Here is another example:

```
x = 42  if x
> 0:
   if x < 100:
      print("x is positive and less than 100")     else:
      print("x is positive and greater than or equal to 100") else:
   print("x is negative or zero")
```

The if-elif-else Statement

The if-elif-else statement in Python allows you to specify multiple conditions and take different actions depending on which condition is met.

```
if condition_1:
statement  statement
etc. elif condition_2:
 statement  statement
etc. else:
 statement  statement
etc.
```

Here is an example: x =

```
42 if x < 0:
   print("x is negative") elif x == 0:
   print("x is zero") else:
```

```
        print("x is positive")
```

In this example, x is compared to 0 first. If x is less than 0, the first branch is taken and the message "x is negative" is printed. If x is equal to 0, the second branch is taken and the message "x is zero" is printed. If x is greater than 0, the final branch is taken and the message "x is positive" is printed.

Note that the elif clause is optional, and you can have multiple elif clauses. When a condition is met, the corresponding branch is executed and the remaining conditions are skipped.

Here is another example:

```
    if score < 60:          print ('Your grade is
    F.') elif score < 70:
            print ('Your grade is D.')  elif score < 80:
            print ('Your grade is C.')  elif score < 90:
            print ('Your grade is B.')
else:
           print ('Your grade is A.')
```

3.6 Logical Operators

Operator	Meaning
and	The and operator connects two Boolean expressions into one compound expression. Both subexpressions must be true for the compound expression to be true.
or	The or operator connects two Boolean expressions into one compound expression. One or both subexpressions must be true for the compound expression to be true. It is only necessary for one of the subexpressions to be true, and it does not matter which.
not	The not operator is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The not operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true.

Boolean Compound Boolean expressions using logical operators

Expression	Meaning
`x > y and a < b`	Is x greater than y AND is a less than b?
`x == y or x == z`	Is x equal to y OR is x equal to z?
`not (x > y)`	Is the expression x > y NOT true?

3.6.1 The and Operator

Expression	Value of the Expression
true and false	false
false and true	false
false and false	false
true and true	true

Example: x =

5 y =

10

if x < 10 and y > 5:

print("Both conditions are True") else:

print("One of the conditions is False")

Output

Both conditions are True

In the above example, both x < 10 and y > 5 are True, so the expression x < 10 and y > 5 is True. This means that both conditions are True and the if block gets executed.

Another example:

if temperature < 20 and minutes > 12:

print 'The temperature is in the danger zone.'

3.6.2 The or Operator

Expression	Value of the Expression
true or false	true
false or true	true
false or false	false
true or true	true

Example: x = 5 y = 10 if x < 10

```
or y < 5:
   print("At least one condition is True")
else:
   print("Both conditions are False")
```

Output

At least one condition is True

In the above example, x < 10 is True and y < 5 is False, so the expression x < 10 or y < 5 is True. This means that at least one condition is True and the if block gets executed.

3.6.3 The Not operator

The not operator is a logical operator that negates the truth value of a boolean expression. It returns True if the expression is False, and False if the expression is True.

For example:

```
x = 5 if not (x < 10):
   print("x is not less than 10") else:
   print("x is less than 10")
```

Output

x is not less than 10

Here is another example for a program that prints a message if temperature is not greater than 100 degrees.

```
if not(temperature > 100):
        print (‘This is below the maximum temperature.’)
```

3.6.4 Checking Numeric Ranges with Logical Operators

In Python, logical operators <, >, <=, >=, ==, and != are used to compare values and check if a number falls within a specific range.

The following four examples all check if a number is within a specific range:

Example 1:

```
if x >= 20 and x <= 40:
        print (‘The value is in the acceptable range.’)
```

Example 2:

```
if x < 20 or x > 40:
        print (‘The value is outside the acceptable range.’)
```

Example 3:

```
# This is an error! if x < 20
and x > 40:
        print (‘The value is outside the acceptable range.’)
```

Example 4:

```
num = 10 if num >= 0 and num <=
10:
   print("Number is within the range 0 to 10") else:
   print("Number is outside the range")
```

Refer to the prescribed textbook for more examples.

3.7 Boolean Variables

A boolean variable is a variable that can take on either of two values: True or False. These values are often used to represent the truth value of a condition in a program.

Here is an example: a = True b = False

```
print(a)   # Output: True print(b)   # Output: False Boolean variables are often used in control structures, such as if statements, to determine the flow of the program based on whether a certain condition is true or false:
flag = True if flag:
   print("Flag is set to True") else:
   print("Flag is set to False")
```

Here is another example:

```
if sales >= 50000:
        sales_quota_met = True else:
                sales_quota_met = False
```



Review Questions

Multiple Choice

1. A _____ structure can execute a set of statements only under certain circumstances.

a. sequence

b. circumstantial

c. decision

d. Boolean

2. A _____ structure provides one alternative path of execution.

a. sequence

b. single alternative decision

c. one path alternative

d. single execution decision

3. You use a(n) _____ statement to write a dual alternative decision structure. a. test-jump

b. if

c. if-else

d. if-call

4. and, or, and not are _____ operators.

a. relational

b. logical

c. conditional

d. ternary

5. The _____ operator takes a Boolean expression as its operand and reverses its logical value.

a. and

b. or

c. not

d. either

6. A _____ is a Boolean variable that signals when some condition exists in the program.

a. flag

b. signal

c. sentinel

d. siren

7. You need to test a condition then execute one set of statements if the condition is true.

8. If the condition is false, you need to execute a different set of statements. What structure will you use?

9. Briefly describe how the and operator works.

10. Briefly describe how the or operator works.

11.The following code contains several nested if-else statements. Unfortunately, it was writ ten without proper alignment and indentation. Rewrite the code and use the proper conventions of alignment and indentation. if score >= A_score: print('Your grade is A.')
else:
if score >= B_score:
print('Your grade is B.')
else: if score >=
C_score: print('Your
grade is C.') else: if
score >= D_score:
print('Your grade is D.')
else:
print('Your grade is F.')

12. Write nested decision structures that perform the following: If amount1 is greater than 10 and amount2 is less than 100, display the greater of amount1 and amount2.

13. The area of a rectangle is the rectangle's length times its width. Write a program that asks for the length and width of two rectangles. The program should tell the user which rectangle has the greater area, or if the areas are the same.

14. On a roulette wheel, the pockets are numbered from 0 to 36. The colours of the pockets are as follows:

• Pocket 0 is green.

• For pockets 1 through 10, the odd-numbered pockets are red and the even-numbered pockets are black.

- For pockets 11 through 18, the odd-numbered pockets are black and the even-numbered pockets are red.
- For pockets 19 through 28, the odd-numbered pockets are red and the even-numbered pockets are black.
- For pockets 29 through 36, the odd-numbered pockets are black and the even-numbered pockets are red.

Write a program that asks the user to enter a pocket number and displays whether the pocket is green, red, or black. The program should display an error message if the user enters a number that is outside the range of 0 through 36.

TOPIC 4
REPETITION STRUCTURES



Learning Outcomes:
After studying this topic you should be able to:

- Use the while loop to write a loop controlled program
- Write programs that use a count-controlled loop
- Write a program that calculates a Running Total
- Understand and use sentinels to terminate loops
- Validate input using loops
- Implement programs that use nested loops

4.1 Introduction

Repetition structures, also known as looping structures, allow you to repeat a set of instructions multiple times. There are two main types of looping structures in Python:
for loops and while loops.

4.2 Introduction to Repetition Structures

Repetition structures (also known as loops) allow you to execute a block of code multiple times. There are two types of loops in Python: for loops and while loops.

A for loop is used to iterate over a sequence (such as a list, tuple, or string) or other iterable object, and execute a block of code for each item in the sequence. A while loop is used to repeatedly execute a block of code as long as a given condition is true.

Condition-Controlled and Count-Controlled Loops
A true/false condition is used in a condition-controlled loop to limit how many times it iterates.
A count-controlled loop only loops back a predetermined amount of times.

4.3 The while Loop: a Condition-Controlled Loop

The while Loop checks its condition before beginning an iteration because it is a Pretest Loop.
General Format while

```
    condition:
        statement
```

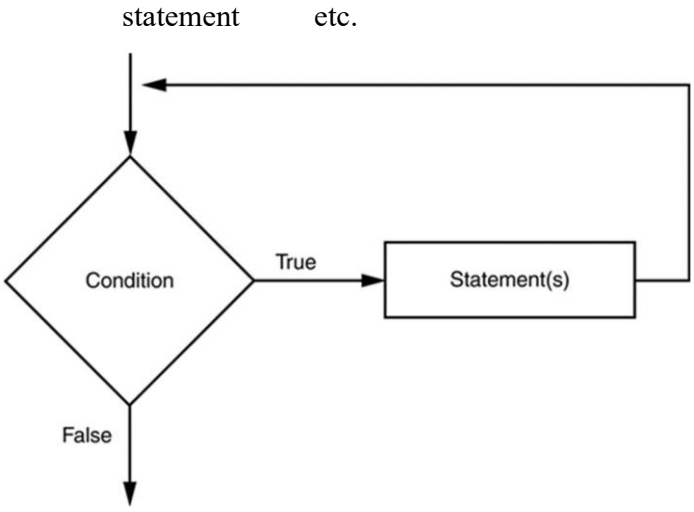


Figure 4-1: The logic of a while loop

Source: Gaddis (2021) Page 171

Example:

```
count = 0 while
count < 5:
print(count)   count
+= 1
```

Output

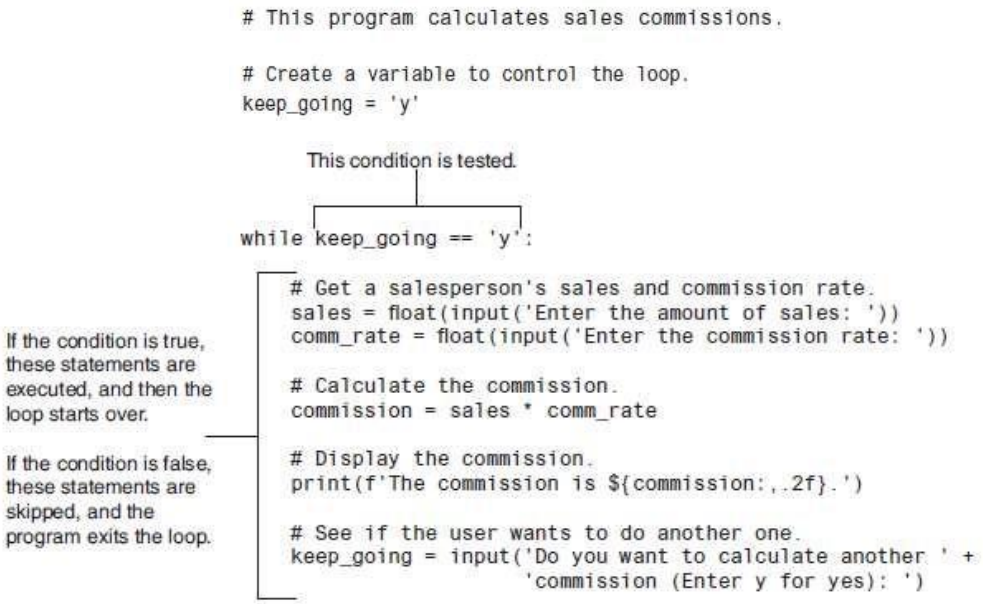
```
0
1
2
3
4
```

The following steps explain the code for the above program.

1. The variable count is initialized to 0.
2. The while loop starts with the condition count < 5. As long as this condition is true, the loop will continue to execute.
3. The first iteration of the loop prints the value of count, which is 0.
4. After printing, the value of count is incremented by 1 using count += 1.

5. The condition count < 5 is evaluated again. Since it's still true (0 < 5), the loop continues to execute.
6. The second iteration of the loop prints the value of count, which is now 1.
7. The value of count is incremented again by 1.
8. The condition count < 5 is evaluated again. Since it's still true (1 < 5), the loop continues to execute.
9. This process repeats until the value of count becomes 5, at which point the condition count < 5 is no longer true, and the loop terminates.

The following example is found on page 172 of the prescribed textbook.




```
# This program calculates sales commissions.

# Create a variable to control the loop.
keep_going = 'y'

while keep_going == 'y':
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate

    # Display the commission.
    print(f'The commission is ${commission:,.2f}.')

    # See if the user wants to do another one.
    keep_going = input('Do you want to calculate another ' +
                       'commission (Enter y for yes): ')
```

Something must happen inside the loop to change the condition keep_going == 'y' to false in order for the loop to terminate. The last statement in this example takes care of that. The loop will terminate if the user enters a value which is not ‘y’; in this case the condition will evaluate to a false.

4.4 The for Loop

The for loop in Python is a count-controlled loop, which means that it executes a block of code a certain number of times. The for loop uses a counter to keep track of how many times the loop has been executed and terminates when the counter reaches a specified value. for variable in [value1, value2, etc.]: statement statement etc.

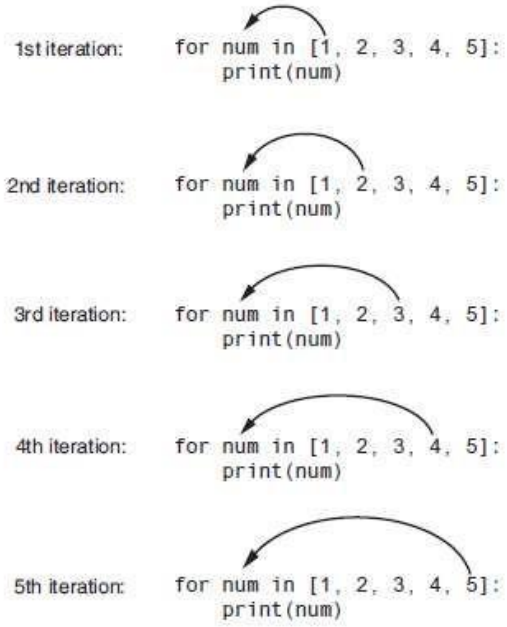
```
for num in [1, 2, 3, 4,5]:      print (num)
```

Output

```
1
2
3
4
5
```

The above code will print the numbers 1 through 5 on separate lines.

Here is an explanation of this code:



4.4.1 Using the range Function with the for Loop

The syntax of the for loop using the range function is as follows:

```
for variable in range(start, end, step):
    statement statement    etc. variable is a variable that takes on the value
```

of the current iteration of the loop.

range is a built-in function that generates a sequence of numbers. The start argument specifies the starting value of the sequence, the end argument specifies the stopping value (which is not included in the sequence), and the step argument specifies the difference between each number in the sequence. If step is not specified, it defaults to

1.

Example:

```
for num in range(1, 10, 2):    print (num)
```

Output

```
1
3
5
7
9
```

- First argument, 1, is the starting value for the list
- Second argument, 10, is the ending limit of the list
- Third argument, 2, is the step value

If one argument is passed to the range function, as shown in the above program, that argument is used as the ending limit of the sequence of numbers.

```
for num in range(5):          print
(num)
```

Output

```
0
1
2
3
```

4

If you provide the range function two arguments, the first argument serves as the sequence's starting value and the second argument serves as its ending limit. For example:

```
for num in range(1, 5):
    print(num)
```

```
Output
1
2
3
4
```

Here is another example:

```
for num in range(5):
    print('Dings')
```

```
Output
Dings   Dings
Dings
Dings
Dings
```

The following program uses a loop to display a table showing the numbers 1 through 10 and their squares.

```
# Print the table headings.  print('Number\tSquare')  print('-------
-------')
```

```
# Print the numbers 1 through 10  # and their
squares.
for number in range(1, 11):  square =
        number**2
        print(f'{number}\t{square}')
```

Output

```
Number          Square
---------------
1               1
2               4
3               9
4               16
5               25
6               36
7               49
8               64
9               81
10              100
```



Think Point: How would you modify the above program so as to let the use control the loop iterations by asking the user to enter the start, end and step values?

- The range function generates a sequence of numbers from 1 to 10 (range(1, 11)).
- The for loop iterates over this sequence, with the loop variable number taking on each value in turn.
- In each iteration, the loop calculates the square of number by raising it to the power of 2 (number**2), and stores the result in the square variable.
- Finally, the loop uses an f-string to print the number and its square, separated by a tab (\t) character, on a single line.

4.4.2 Generating Lists that Range from Highest to Lowest for num

```
in range(5, 0, -1):  print (num)
```

Output
5
4
3
2
1

- The range function generates a sequence of numbers from 5 to 1 (range(5, 0, -1)), counting down by 1 with each step (the third argument -1).
- The for loop iterates over this sequence, with the loop variable num taking on each value in turn.
- In each iteration, the loop uses the print function to print the value of num on a new line.



Think Point: What are the disadvantages of duplicated code resulting from not using loops?

4.5 Calculating a Running Total

One can calculate a running total using a for loop by initializing a variable to store the total and updating it in each iteration of the loop. Typically, two components are used in programs that determine the sum of a set of numbers:

- A loop for reading each number in the sequence
- A variable that tally's the numbers as they are read.

Here is an example that calculates the running total of a list of numbers: num1 = 5 num2 = 3 num3 = 7 num4 = 2 num5 = 8 running_total = 0

```
for number in [num1, num2, num3, num4, num5]:
    running_total += number
print(f'The total is {total}.')
```

Output

25

- The variables num1, num2, num3, num4, and num5 store the numbers to be summed.
- The running_total variable is initialized to 0 to store the total.
- The for loop iterates over the list of numbers [num1, num2, num3, num4, num5], with the loop variable number taking on each value in turn.
- In each iteration, the loop adds the current number to the running_total

- Note that the running_total is updated in each iteration to accumulate the sum of all the numbers in the list. The final value of running_total is 25, which is the sum of all the numbers in the list.

Refer to page 189 of the prescribed textbook for a similar example for calculating a running total.

4.6 The Augmented Assignment Operators

Augmented assignment operators are a shorthand notation in programming languages for updating the value of a variable. They perform the operation and assignment of the result to the variable in a single statement. For example, instead of writing "x = x + 1", the augmented assignment operator "+=" can be used to write "x += 1"

Operator	Example Usage	Equivalent To
+=	x += 5	x = x + 5
-=	y -= 2	y = y - 2
*=	z *= 10	z = z * 10
/=	a /= b	a = a / b
%=	c %= 3	c = c % 3
//=	x //= 3	x = x // 3
**=	y **= 2	y = y**2

Source: Gaddis (2021) page 191

4.7 Sentinels

A sentinel in Python is a special value used to indicate a specific condition, such as the end of a data stream or the termination of a loop. Sentinels are often used as markers to determine when a certain action should be performed, such as breaking out of a loop or returning a specific value. In Python, the None object is a commonly used sentinel value, indicating the absence of a value.

Example:

The county tax office calculates the annual taxes on property using the following formula: property tax = property value X 0.0065

You learn that each property is assigned a lot number, and all lot numbers are 1 or greater. You decide to write a loop that uses the number 0 as a sentinel value. During each loop iteration, the program will ask the clerk to enter either a property's lot number, or 0 to end.

```
# This program displays property taxes.
TAX_FACTOR = 0.0065   # Represents the tax factor.
# Get the first lot number.
        print('Enter the property lot number or enter 0 to end.')
lot = int(input('Lot number: '))

# Continue processing as long as the user  # does not enter
lot number 0.
while lot != 0:
    # Get the property value.  value = float(input('Enter the property
    value: '))

    # Calculate the property's tax.
    tax = value * TAX_FACTOR

    # Display the tax.  print(f'Property tax:
    ${tax:,.2f}')

    # Get the next lot number.
    print('Enter the next lot number or enter 0 to end.')  lot = int(input('Lot
    number: '))
```

Output

Program Output (with input shown in bold)

Enter the property lot number or enter 0 to end.

Lot number: 100

Enter the property value: 100000.00

Property tax: $650.00. Enter the next

lot number or enter 0 to end.

Lot number: 200

Enter the property value: 5000.00 Property tax:

$32.50.

Enter the next lot number or enter 0 to end.

Lot number: 0



Research: How can you use a loop for input validation? Use an example of a relevant Python program to explain this concept

4.9 Nested Loops

Nested loops in Python refers to the usage of one or more loops inside another loop. The inner loop runs to completion for each iteration of the outer loop. This allows for the processing of a series of elements in a series of nested iterations. The syntax for a nested loop in Python is:

```
for outer_loop_variable in outer_loop_iterable:
    # code in the outer loop     for inner_loop_variable in
inner_loop_iterable:
        # code in the inner loop
```

The outer_loop_variable and inner_loop_variable can be any variable name of your choice. For each iteration of the outer loop, the inner loop goes through all of its iterations. Inner loops complete iterations before outer loops.

Example:

```
for i in range(1, 11):     for j in range(1,
11):         print(i * j, end='\t')     print()
```

Output

```
1    2    3    4    5    6    7    8    9    10
2    4    6    8    10   12   14   16   18   20 3    6
     9    12   15   18   21   24   27   30 4  8    12
     16   20   24   28   32   36   40 5  10   15   20
     25   30   35   40   45   50
6    12   18   24   30   36   42   48   54   60
7    14   21   28   35   42   49   56   63   70
8    16   24   32   40   48   56   64   72   80
9    18   27   36   45   54   63   72   81   90
10   20   30   40   50   60   70   80   90   100
```

This code uses two nested for loops. The outer loop is controlled by the variable i and iterates over the range (1, 11). The inner loop is controlled by the variable j and iterates over the range (1, 11). For each iteration of the outer loop, the inner loop runs to completion, resulting in the multiplication of i and j for each iteration, which is printed on the same line separated by a tab. After each inner loop iteration, a newline character is printed to move to the next line. The output of this code would be a table of the multiplication of integers from 1 to 10.

outer loop: i = 1 inner
loop: j = 1 i * j = 1 inner
loop: j = 2 i * j = 2
  ...
  inner loop: j = 10 i * j = 10
outer loop: i = 2 inner loop: j
= 1 i * j = 2 inner loop: j =
2 i * j = 4
  ...
  inner loop: j = 10 i * j = 20
  ...

outer loop: i = 10 inner

loop: j = 1 i * j = 10 inner

loop: j = 2 i * j = 20

...

inner loop: j = 10 i * j = 100



Problem: Write a program that displays a rectangular pattern of asterisks and a triangle of asterisks using nested loops.



Review Questions

Multiple Choice

1. A _____ -controlled loop uses a true/false condition to control the number of times that it repeats.

a. Boolean

b. condition

c. decision

d. count

2. A _____ -controlled loop repeats a specific number of times.

a. Boolean

b. condition

c. decision

d. count

3. The -= operator is an example of a(n) _____ operator.

a. relational

b. augmented assignment

c. complex assignment

d. reverse assignment

4. A(n) _____ variable keeps a running total.

a. sentinel

b. sum

c. total

d. accumulator

5. The input operation that appears just before a validation loop is known as the . a. prevalidation read

b. primordial read

c. initialization read

d. priming read

6. Validation loops are also known as .

a. error traps

b. doomsday loops

c. error avoidance loops

d. defensive loops

7. True/False

To calculate the total number of iterations of a nested loop, add the number of iterations of all the loops.

8. True/False

The process of input validation works as follows: when the user of a program enters invalid data, the program should ask the user "Are you sure you meant to enter that?" If the user answers "yes," the program should accept the data.

9. What is an infinite loop? Write the code for an infinite loop.
10. Why is it critical that accumulator variables are properly initialized?
11. Why must the value chosen for use as a sentinel be carefully selected?
12. Write a while loop that asks the user to enter two numbers. The numbers should be added and the sum displayed. The loop should ask the user if he or she wishes to perform the operation again. If so, the loop should repeat, otherwise it should terminate.
13. Write code that prompts the user to enter a number in the range of 1 through 100 and validates the input.
14. Write a program that uses nested loops to collect data and calculate the average rainfall over a period of years. The program should first ask for the number of years. The outer loop will iterate once for each year. The inner loop will iterate twelve times, once for each month. Each iteration of the inner loop will ask the user for the inches of rainfall for that month. After all iterations, the program should display the number of months, the total inches of rainfall, and the average rainfall per month for the entire period.

TOPIC 5
FUNCTIONS



Learning Outcomes:

After studying this topic you should be able to:

- Gain knowledge of writing and calling void functions

- Define and all a void function
- Design a program to use functions
- Declare and use local variables
- Pass arguments to functions
- Declare and use global variables and global constants
- Define and call value-returning functions
- Use the math module
- Store functions in modules

5.1 Introduction

A function in Python is a reusable block of code that performs a specific task. Functions are defined using the "def" keyword, followed by the function name and a set of parentheses. The input parameters (if any) are specified within these parentheses. The code within the function is indented and executed when the function is called.

Functions can optionally return a value using the "return" statement.

5.1.1 Divide and Conquer

Divide and conquer is a popular algorithm design paradigm that involves breaking down a problem into smaller, subproblems and solving each of them individually. The solutions of the subproblems are then combined to solve the original problem. In Python, divide and conquer algorithms can be implemented using functions.

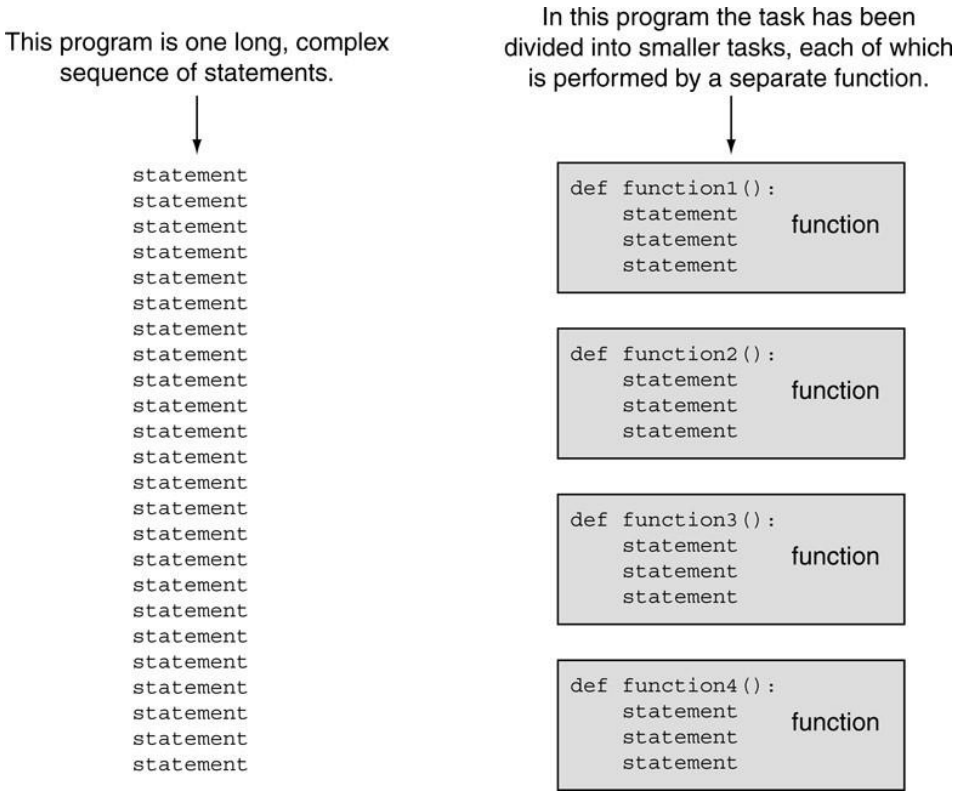


Figure 5-1 Using functions to divide and conquer a large task

Source: Gaddis(2021) page 220

5.1.2 Benefits of Using Functions

Modularizing a program with functions has several benefits including:

- Improved readability and maintainability: Functions allow for breaking down a complex program into smaller, manageable chunks, making the code easier to read, understand, and maintain.

- Reusability: Functions can be reused across multiple parts of the program or even in other programs, reducing the amount of code that needs to be written and making it easier to test and debug.

- Encapsulation: Functions allow for encapsulating implementation details and exposing only the necessary information to the rest of the program, promoting modularity and separation of concerns.

- Better error handling: Functions can be designed to return error codes or throw exceptions, making it easier to handle errors and exceptions within the program.

- Improved testing: Modularizing a program with functions allows for testing individual components in isolation, making it easier to identify and fix bugs.

5.2 Defining and Calling a Function

Function Names

- No Python key words allowed
- Contains no spaces
- The first character must be an underscore (_) or one of the letters a through z.
- Use the letters a through z, A through Z, numerals 0 through 9, or underscore after the first character.
- Characters in uppercase and lowercase are different.

Examples of good function names include get_pay, get_gross_amount, calculate_overtime, print_receipt, and so on.

Defining and Calling a Function

In Python, you can define a function using the "def" keyword, followed by the function name and a set of parentheses that may contain parameters.

```
def function_name():                    #function header statement
        statement etc.
```

This is the basic syntax of a function in Python.

- "function_name" is the name of the function, which should be descriptive and follow the naming conventions for variables in Python (lowercase with words separated by underscores).
- "statement" represents one or multiple lines of code that perform a specific task. These statements are executed when the function is called.
- The code inside the function is indented to indicate that it is part of the function. The function is defined using the "def" keyword and ends with a colon (:). The statements

inside the function are executed when the function is called. To call the function, simply write the function name followed by parentheses "()".

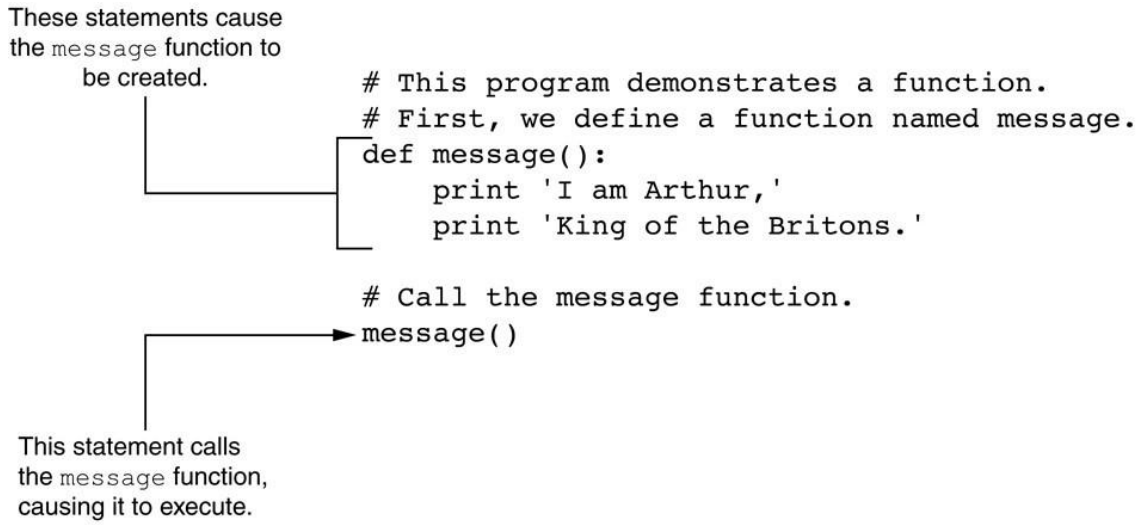


Figure 5-2 The function definition and the function call

Source: Gaddis(2021) page 224

5.2.1 Indentation in Python

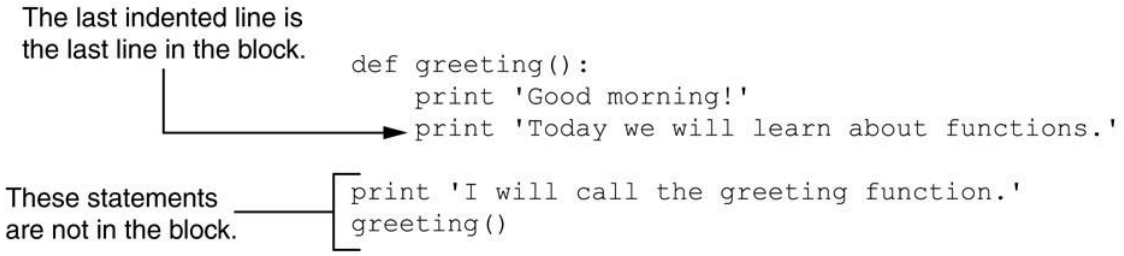


Figure 5-3 All of the statements in a block are indented

Source: Gaddis(2021) page 224

5.2.2 Void Function

A void function in Python is a function that does not return a value. It is defined using the def keyword, followed by the function name and parameters, and concluded with the return statement without any value or with the pass statement. A void function can perform operations, print results, and change the state of its arguments, but it does not return a value to be assigned to a variable or used as an expression.

Example of a void function:

```
def greet(name):
    print("Hello, " + name)
greet("Dings")
```

Output

Hello, Dings

In this example, the greet function takes a parameter name and prints a greeting message using it, but it does not return a value.

Here is an example of a program with two functions: main and message.

```
1  # This program has two functions. First we
2  # define the main function.
3  def main():
4      print('I have a message for you.')
5      message()
6      print('Goodbye!')
7
8  # Next we define the message function.
9  def message():
10     print('I am Arthur,')
11     print('King of the Britons.')
12
13 # Call the main function.
14 main()
```

Source: Gaddis (2021) Page 225

The definition of the main function appears in lines 3 through 6, and the definition of the message function appears in lines 9 through 11. The statement in line 14 calls the main function.

The interpreter jumps to the `main` function and begins executing the statements in its block.

```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

The interpreter jumps to the `message` function and begins executing the statements in its block.

```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

```
# This program has two functions. First we
# define the main function.
def main():
    print('I have a message for you.')
    message()
    print('Goodbye!')

# Next we define the message function.
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# Call the main function.
main()
```

When the main function ends, the interpreter jumps back to the part of the program that called it. There are no more statements, so the program ends.



NOTE: When a program calls a function, the function takes control of the program's execution. This means that the control of the program simply transfers to that function.

5.2.3 Pausing Execution Until the User Presses Enter

In Python, you can use the input function to cause a program to pause until the user presses the Enter key.

```
input('Press Enter to see Step 1.')
```

This statement displays the prompt 'Press Enter to see Step 1.' and pauses until the user presses the Enter key.

Using the pass Keyword

In Python, the pass keyword is used as a placeholder for code that has not been implemented yet. It is also used when a statement is required syntactically but you do not want any command to execute.

For example, consider the following code:. The pass keyword is ignored by the Python interpreter, so the following code creates four functions that do nothing.

```
def step1(): pass
def step2(): pass
def step3(): pass
def step4():
        pass
```



Think Point: What is the importance of the pass keyword in program development?

5.4 Local Variables

Variables declared inside a function are called local variables. They can only be accessed within the function in which they are declared and are not accessible from outside the function. Here's an example program to demonstrate the use of local variables in Python:

```
def first_function():      local_variable = 10
    print("Inside first_function:", local_variable)
 def second_function():
local_variable = 20
    print("Inside second_function:", local_variable)

first_function() second_function()
print("Outside both functions:", local_variable)  # This line will raise an error
```

Output

```
Inside first_function: 10
Inside second_function: 20
Outside both functions: NameError: name 'local_variable' is not defined
```

As you can see, the local variable local_variable is defined inside both functions first_function and second_function. However, each function has its own separate instance of local_variable and changes to local_variable inside one function do not affect the value of local_variable in the other function.

Also, when we try to access local_variable outside of both functions, a NameError is raised, as the variable is not defined in the global scope. The scope of variable refers to the region of the code where the variable is defined and can be accessed. In other words, it's the area of the code where the variable has visibility and can be used.

Local variables are declared inside a function and are only accessible within the function in which they are declared. They are not accessible from outside the function, even from other functions in the same program.

5.5 Passing Arguments to Functions

An argument is any piece of data that is passed into a function when the function is called. A parameter is a variable in a function definition that is used to receive values from the caller of the function. When a function is called, the values passed as arguments are assigned to the corresponding parameters in the function definition.

Example:

```
# This program demonstrates an argument being # passed to a
function.
def main():
    value = 5        show_double(value)
# The show_double function accepts an argument  # and displays
double its value.
def show_double(number):            result
= number * 2        print(result)
# Call the main function.
main()
```

Output

10

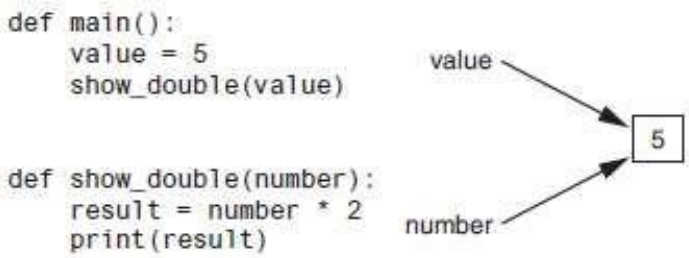
- The code above defines a Python program with two functions: main and show_double.

- The main function is the entry point of the program, and it defines a local variable named value with the value 5.

- The main function then calls the show_double function and passes the value variable as an argument.

```
def main():
    value = 5
    show_double(value)


def show_double(number):
    result = number * 2
    print(result)
```



The value variable and the number parameter reference the same value.

- The show_double function takes one argument, number, and multiplies it by 2 to get the result.

- The show_double function then prints the result.

- Finally, the main function is executed, which triggers the execution of the show_double function.

5.5.1 Parameter Variable Scope

The function in which a parameter variable is used is referred to as the parameter's scope. The parameter variable can be accessed from statements inside the function, but not from statements outside the function.

5.5.2 Passing Multiple Arguments def

```
calculate(a, b):      result = a + b
print(result)
 def main():
calculate(3, 5) main()
```

Output

8

Positional Arguments: These are the most common type of arguments, where the values are passed to the function in the same order as they are defined. The values are then assigned to the corresponding parameters in the function definition.

Example:

```
def greet(name, message):
   print("Hello, " + name + ". " + message)

greet("John", "How are you doing?")
```

Output

Hello, John. How are you doing?

Keyword Arguments: These are the arguments that are passed using the parameter name, followed by an equal sign and its corresponding value. This way, the order of the arguments does not matter.

Example:

```
def greet(name, message):
   print("Hello, " + name + ". " + message) greet(message="How are you doing?",
name="John")
```

Output

Hello, John. How are you doing?

Default Arguments: These are the arguments that have a default value in the function definition. If these arguments are not passed when the function is called, their default value is used.

Example:

def greet(name, message="Good morning!"): print("Hello, " + name + ". " + message)

greet("John")

Output

Hello, John. Good morning!

```
1  # This program demonstrates a function that accepts
2  # two arguments.
3
4  def main():
5      print('The sum of 12 and 45 is')
6      show_sum(12, 45)
7
8  # The show_sum function accepts two arguments
9  # and displays their sum.
10 def show_sum(num1, num2):
11     result = num1 + num2
12     print(result)
13
14 # Call the main function.
15 main()
```

Program Output

```
The sum of 12 and 45 is
57
```

Source: Gaddis (2021) Page 240

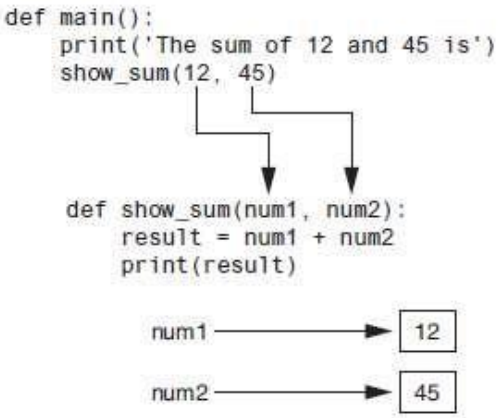


Figure 5-4 Two arguments passed to two parameters

Source: Gaddis(2021) page 241



Think Point: What would be the effect of reversing the order of arguments passed to a function when it is called?

When an argument is passed to a function in Python, the function parameter variable will reference the argument's value. However, any changes that are made to the parameter variable will not affect the argument.

For example,

```
    #This program demonstrates what happens when you  # change the
    value of a parameter.
def main():
            value = 99
            print(f'The value is {value}.')  change_me(value)
                    print(f'Back in main the value is {value}.')
             def change_me(arg):  print('I am changing
    the value.')  arg = 0
            print(f'Now the value is {arg}.')
```

```
# Call the main function.
main()
```

Output

```
The value is 99.
I am changing the value.
Now the value is 0.
Back in main the value is 99.
```

In the code, you are passing the value 99 to the function change_me. The function change_me receives the argument arg and assigns it a new value 0. However, this change to the arg value does not affect the original value of value passed to the function. Hence, when you print the value of value after calling the change_me function, it still shows the original value of 99. The form of argument passing that is used in Python, where a function cannot change the value of an argument that was passed to it, is commonly called pass by value.

5.5.3 Mixing Keyword Arguments with Positional Arguments

In Python, you can mix keyword arguments (also known as named arguments) with positional arguments when calling a function. Keyword arguments are arguments that are passed to a function by explicitly specifying the name of the argument and its value, while positional arguments are passed to a function by their position in the argument list.

Example:

```
# This program demonstrates keyword arguments.
def main():
    # Show the amount of simple interest, using 0.01 as
    # interest rate per period, 10 as the number of periods,  # and $10,000 as the
    principal.
    show_interest(rate=0.01, periods=10, principal=10000.0)

# The show_interest function displays the amount of
# simple interest for a given principal, interest rate  # per period, and
number of periods.
```

```
def show_interest(principal, rate, periods):
        interest = principal * rate * periods
        print(f'The simple interest will be ${interest:,.2f}.')

# Call the main function.
main()
```

Output

The simple interest will be $1000.00.

In this case, the first argument principal is passed as a positional argument, followed by rate and periods which are passed as keyword arguments.



Problem: Explain whether the following function call will cause an error for the above program:

show_interest(1000.0, rate=0.01, 10)



Think Point: When are keyword arguments useful?

5.6 Global Variables and Global Constants

5.6.1 Global Variables

In Python, global variables are variables that are defined outside of any function and can be accessed from anywhere in the code, including inside functions. Global variables are defined using the global keyword. For example:

```
x = 10 def
increase_x():
global x      x += 1
print(x)
increase_x ()         # prints 11
```

5.6.2 Global constants

Global constants, on the other hand, are global variables that have a fixed value and cannot be changed once set. The naming convention for global constants is to use all capital letters with underscores between words. For example:

```
PI = 3.14

def display_pi():      print(PI)


display_pi () # prints 3.14
```

It is important to note that, even though global constants are named with all capital letters, there is no actual enforcement in Python to prevent them from being modified. It is just a convention used to indicate that their value should not be changed. To enforce immutability, you can use the constant module or a similar library.

Here is another example that uses a global variable and a global constant:

```
# Global constant
PI = 3.14  def
calc_circumference(radius):
    # Accessing the global constant      circumference = 2 *
PI * radius      return circumference


# Global variable circumference = 0


# Using the function to calculate the circumference circumference =
calc_circumference(10)


print(circumference)
```

For the following reasons, limit the use of global variables:

- Debugging is challenging when using functions that depend on global variables.
- Functions that use global variables are usually dependent on those variables.

- A program is difficult to understand when it uses global variables.

5.7 Introduction to Value-Returning Functions

A value-returning function in Python is a function that returns a value to the caller. The return statement is used to return a value from the function. def function_name(): statement statement etc.

return expression

Here is a simple example of a value-returning function:

```
# This program uses the return value of a function.
def main():
    # Get the user's age.
    first_age = int(input('Enter your age: '))

    # Get the user's best friend's age.  second_age = int(input("Enter your
    best friend's age: "))

    # Get the sum of both ages.
    total = sum(first_age, second_age)

    # Display the total age.  print(f'Together you are {total}
    years old.')

# The sum function accepts two numeric arguments and  # returns the
sum of those arguments.
def sum(num1, num2):  result =
    num1 + num2  return result

# Call the main function.
main()
```

Output (with input shown in bold)

Enter your age: 22

Enter your best friend's age: 24 Together you

are 46 years old.

- This code will run without any errors and will provide the expected output.
- When the program is run, it will prompt the user to enter their age and their best friend's age. The inputs will be stored as the variables first_age and second_age.
- Then, the program calls the sum function and passes first_age and second_age as arguments. The sum function adds the two arguments together and returns the result, which is stored in the variable total.

Finally, the program displays the message Together you are [total] years old., where [total] is the sum of the ages of the user and their best friend.

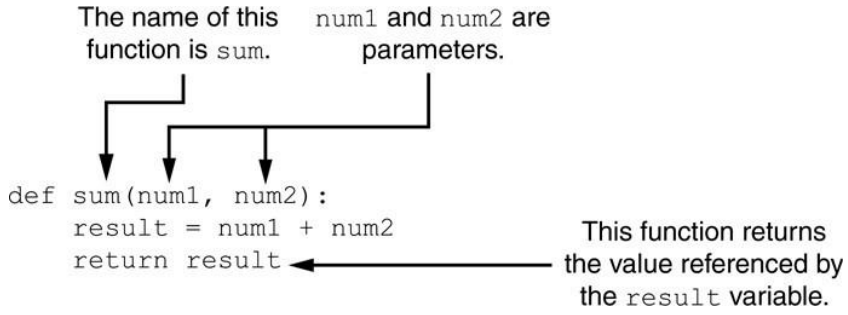


Figure 5-5 Parts of the function

Source: Gaddis (2021) page 261

Example:

```
def square(x):      return x *
x


result = square(5)
print(result)
```

In this example, the square function takes a single argument x and returns the square of that value. The return value of the function is assigned to the result variable, which is then printed to the screen. The output of this program will be 25.

It is important to note that a function can only return one value. If you want to return multiple values, you can use a tuple, a list, or another type of collection to store them.

The following is another example of a value-returning function:

```
 def sum_of_squares(x, y):      sum =
x**2 + y**2      return sum
result = sum_of_squares(3, 4) print("The sum of squares
is:", result)
```

5.7.1 Standard Library Functions

These functions are already written and come with the language. They make the job of a programmer easier and perform a variety of frequently required tasks. These functions are stored in files known as modules. To use the standard library functions, you include the import statement near the beginning of the program.

Because you do not see the internal workings of library functions, many programmers think of them as black boxes. The term “black box” is used to describe any mechanism that accepts input, performs some operation (that cannot be seen) using the input, and produces output.

For example: input() , range()



Generating Random Numbers

```
# This program displays five random  # numbers in the
range of 1 through 100.  import random
 def main():   for count in range(5):
                        print(random.randint(1, 100))
```

```
# Call the main function.  main()
```

- The program generates and displays five random numbers in the range of 1 to 100.
- It first imports the random module, which provides functions for generating pseudorandom numbers. Then, the main function is defined, which contains a for loop that runs five times and generates a random integer in the range of 1 to 100 using the random.randint function. Finally, the main function is called to execute the program.
- Each time you run the program, it will generate a different set of five random numbers.

5.7.2 Using Value-Returning Functions

Example

```
def calculate_salary(basic_salary, allowances):
    # Calculate gross salary
    gross_salary = basic_salary + allowances

    # Calculate the tax amount     tax = 0.1 *
gross_salary

    # Calculate net salary     net_salary =
gross_salary - tax     # Return both gross and net
salaries     return gross_salary, net_salary
def main():
    # Get the basic salary from the user     basic_salary = float(input("Enter
your basic salary:
"))

    # Get the allowances from the user
    allowances = float(input("Enter your allowances: "))

    # Call the calculate_salary function
        gross_salary,                    net_salary                    =
```

```
calculate_salary(basic_salary, allowances)

    # Display the results
    print(f"Gross Salary: {gross_salary}")     print(f"Net Salary:
{net_salary}")

# Call the main function main()
```

- When the program is run, it will prompt the user to enter their basic salary and allowances. The inputs will be stored as the variables basic_salary and allowances.
- Then, the program calls the calculate_salary function and passes basic_salary and allowances as arguments. The calculate_salary function calculates the gross salary, tax amount, and net salary based on the inputs, and returns both the gross salary and net salary.Finally, the program displays the gross salary and net salary.

5.7.3 Returning Strings

```
def get_name():
    # Get the user's name     name = input("Enter
your name:")
    # Return the name     return name
# Call the get_name function and capture the result result = get_name()

# Display the result print(result)
```

- This code defines a function named get_name() that prompts the user to enter their name and returns the name as the result.
- The function uses the input function to prompt the user to enter their name, which is then stored in the name variable.
- Finally, the function uses the return statement to return the name variable as the result of the function.
- The code then calls the get_name function and captures its result in the result variable. The result is then displayed on the screen using the print function.

- When this code is run, the user will be prompted to enter their name, and the entered name will be displayed on the screen.

5.7.4 Returning Boolean Values

A function can also return a Boolean value, which is either True or False. Boolean values are often used in programming to represent the result of a comparison or a test.

```
def is_positive(number):     if
        number > 0:         return True
        else:
            return False
result = is_positive(10) print(result) # outputs True
result = is_positive(-10) print(result) # outputs
False
```

- In this example, the function is_positive takes a single argument number and returns True if the number is greater than zero, and False otherwise.
- The function is called twice and the result is captured in the result variable. The value of the result variable is then displayed on the screen using the print function.



Think Point: What is the benefit of using value-returning functions?

5.7.5 Returning None from a Function

In Python, you can use the keyword return to specify what value a function should return. If you do not include a return statement in your function, or if you include a return statement with no value, the function will return the value None.

None is a special value in Python that represents the absence of a value. It is often used as a placeholder when a function doesn't need to return a value. For example, a function that just prints some text to the screen might return None to indicate that it doesn't return any meaningful value.

Here is an example of a function that returns None:

```
def my_function():     print("Hello,
World!") result = my_function()
print(result)
```

Output

```
Hello, World!
None
```

The function my_function does not include a return statement, so it returns None by default.

Here is another example that uses the return none statement.

```
def my_function(a, b):     print("The sum of", a, "and", b, "is", a + b)
        return None

result = my_function(3, 4) print(result)
```

Output

```
The sum of 3 and 4 is 7
None
```

The function my_function calculates the sum of its two arguments, prints the result, and then returns None. The value of None is stored in the variable result, and it is printed to the screen.

5.8 The math Module

The math module is a standard module in Python that provides mathematical operations and functions. Some of the common operations that you can perform using the math module are:

1. Basic arithmetic operations: math.ceil(x), math.floor(x), math.trunc(x), math.factorial(x)
2. Exponential and logarithmic functions: math.exp(x), math.log(x), math.log10(x), math.pow(x, y)
3. Trigonometric functions: math.sin(x), math.cos(x), math.tan(x), math.asin(x), math.acos(x), math.atan(x)

4. Constants: math.pi, math.e, math.tau

Here are some of the math module's functions:

math Module Function	Description
`acos(x)`	Returns the arc cosine of `x`, in radians.
`asin(x)`	Returns the arc sine of `x`, in radians.
`atan(x)`	Returns the arc tangent of `x`, in radians.
`ceil(x)`	Returns the smallest integer that is greater than or equal to `x`.
`cos(x)`	Returns the cosine of `x` in radians.
`degrees(x)`	Assuming `x` is an angle in radians, the function returns the angle converted to degrees.
`exp(x)`	Returns e^x
`floor(x)`	Returns the largest integer that is less than or equal to `x`.
`hypot(x, y)`	Returns the length of a hypotenuse that extends from (0, 0) to (`x`, `y`).
`log(x)`	Returns the natural logarithm of `x`.
`log10(x)`	Returns the base-10 logarithm of `x`.
`radians(x)`	Assuming `x` is an angle in degrees, the function returns the angle converted to radians.
`sin(x)`	Returns the sine of `x` in radians.
`sqrt(x)`	Returns the square root of `x`.
`tan(x)`	Returns the tangent of `x` in radians.

Table 5-1 Many of the functions in the math module

Source: Gaddis(2021) page 276

Here is an example of how you can use the math module:

```
import math x
= 10 y = 20
# Logarithmic function result =
math.log(x, 10)

# Power function result =
math.pow(x, y)

# Factorial function result =
math.factorial(x) The Python
```

documentation has more
information about the math module
and this can be access on the
following link:
https://docs.python.org/3/library/math.html

The following is an example of a program that calculates the square root of number:

```
import math
# Calculate the square root of a number x = 100
result = math.sqrt(x)
print("The square root of", x, "is", result)
```

Output

```
The square root of 100 is 10.0
```

- In this example, we start by importing the math module. This gives us access to all the mathematical functions and constants defined in the math module.
- Next, we define a variable x with the value 100. We then use the math.sqrt(x) function to calculate the square root of x.
- The math.sqrt(x) function returns the square root of the argument x as a floatingpoint number. In this case, the square root of 100 is 10.0.

Finally, we print the result using the print function, along with a message indicating what the result represents.



Problem: Write a program that uses other math module functions.

5.8.1 The math.pi and math.e Values

The math module also defines two variables, pi and e, which are assigned mathematical values for pi and e. You can use these variables in equations that require their values. For example, the following statement, which calculates the area of a circle, uses pi.

```
area = math.pi * radius**2
```



Review Questions

1. What is a function?
2. What is meant by the phrase “divide and conquer”?
3. How do functions help you reuse code in a program?
4. A variable that is visible to every function in a program file is a _____. a. local variable

b. universal variable

c. program-wide variable

d. global variable

5. This standard library function returns a random floating-point number in the range of 0.0 up to 1.0 (but not including 1.0).

a. random

b. randint

c. random_integer

d. uniform

6. This statement causes a function to end and sends a value back to the part of the program that called the function.

a. end

b. send

c. exit

d. return

7. This is a math module function.

a. derivative

b. factor

c. sqrt

d. differentiate

8. Why do global variables make a program difficult to debug?
9. Suppose you want to select a random number from the following sequence:

0, 5, 10, 15, 20, 25, 30. What library function would you use?

8. What statement do you have to have in a value-returning function?

9. Look at the following function header: def my_function(a, b, c):

Now look at the following call to my_function:

```
my_function(3, 2, 1)
```

When this call executes, what value will be assigned to a? What value will be assigned to b? What value will be assigned to c?

10. The following statement calls a function named half, which returns a value that is half that of the argument. (Assume the number variable references a float value.) Write code for the function.

```
result = half(number)
```

11. Write a function named times_ten that accepts a number as an argument. When the function is called, it should return the value of its argument multiplied times 10.

12. A county collects property taxes on the assessment value of property, which is 60 percent of the property's actual value. For example, if an acre of land is valued at $10,000, its assessment value is $6,000. The property tax is then 72¢ for each $100 of the assessment value. The tax for the acre assessed at $6,000 will be $43.20. Write a program that asks for the actual value of a piece of property and displays the assessment value and property tax.

13. Write a program that generates a random number in the range of 1 through 100, and asks the user to guess what the number is. If the user's guess is higher than the random number, the program should display "Too high, try again." If the user's guess is lower than the random number, the program should display "Too low, try again." If the user guesses the number, the application should congratulate the user and generate a new random number so the game can start over.

TOPIC 6



FILES

Learning Outcomes:

After studying this topic you should be able to:

- Create files
- Read from files
- Write to files using Python
- Append text to a file
- Use loops to process files

6.1 Introduction

In Python, you can work with files to read from and write to them. This topic introduces sequential file input and output. The learner will gain knowledge of how to read, write, and save vast amounts of data in fields and records.

Terms

- Saving data in a file = "writing data to" the file
- Output file = file that data is written to
- Retrieving data from a file = "reading data from" the file Input file = file that data is read from

6.1 Types of Files

There are two main types of files: text and binary.

Text files: These are files that contain text data, such as a plain text file, a CSV file, or a JSON file. Text files can be opened and read using the built-in open function in Python, and the contents can be processed using string methods.

Binary files: These are files that contain binary data, such as an image, audio, or video file. Binary files can be opened and read using the built-in open function, but the contents must be processed using binary methods, such as reading and writing bytes.

6.1.1 File Access Methods

File access methods refer to the way in which data is retrieved from a file stored on a storage device, such as a hard drive or an SSD. There are two main types of file access methods: sequential and direct.

Sequential Access: This method involves reading the data in a file in a linear, sequential manner, starting from the beginning and working towards the end. Sequential access is often used when reading large files, such as text files, where the contents of the file need to be processed in order. For example, when reading a large text file, a program may read the first line, process the data, then read the next line, and so on. This is similar to the way older cassette tape players work. If you want to listen to the last song on a cassette tape, you have to either fast-forward over all of the songs that come before it or listen to them. There is no way to jump directly to a specific song.

Direct Access: This method involves accessing a specific part of a file without reading the entire file from start to finish. Direct access is often used when working with binary files, such as images or audio files, where specific data needs to be retrieved quickly and efficiently. For example, when reading an image file, a program may seek directly to the part of the file where the image data is stored and read that data, without having to read the entire file. When you work with a direct access file (which is also known as a random access file), you can jump directly to any piece of data in the file without reading the data that comes before it. This is similar to the way a CD player or an MP3 player works. You can jump directly to any song that you want to listen to.

In general, sequential access is used when reading text files or other data that needs to be processed in order, while direct access is used when working with binary files or other data that needs to be retrieved quickly and efficiently. The choice of file access method depends on the specific requirements of the task at hand, and the nature of the data being processed.

6.2 Opening a File

It takes the file name and mode (read, write, or append) as arguments, and returns a file object that you can use to read or write the file.

The open function:

General format:

```
file_variable = open(filename, mode)
```

file_variable is the name of the variable that will reference the file object
filename is a string specifying the name of the file mode is a string
specifying the mode (reading, writing, etc.)

Mode	Description
'r'	Open a file for reading only. The file cannot be changed or written to.
'w'	Open a file for writing. If the file already exists, erase its contents. If it does not exist, create it.
'a'	Open a file to be written to. All data written to the file will be appended to its end. If the file does not exist, create it.

Table 6-1 Some of the Python file modes
Source: Gaddis (2021) page 307

Examples of statements used for opening a file.

customer_file = open(‘customers.txt’, ‘r’)

The code creates a file object for the file customers.txt in read mode ('r'). The open function is used to open a file in Python, and it takes two arguments: the name of the file, and the mode in which the file should be opened. In this case, the mode is 'r', which stands for "read."
The open function returns a file object, which can be used to access the contents of the file.
Here is another example:

sales_file = open(‘sales.txt’, ‘w’)

6.3 Writing Data to a File

Writing to a file: To write to a file, you open it in write mode ("w") or append mode ("a"). In write mode, the contents of the file are overwritten, while in append mode, new data is added to the end of the file. Here's an example that writes a string to a file in write mode:
For example:

file = open("example.txt", "w") file.write("This is some
data.") file.close()

File_variable.write(string)
File_variable is a variable that references a file object.
Write refers to the operation of the file object used to write data.
String refers to the string that will be written to the file.

6.4 Reading Data From a File

Reading a file: To read the contents of a file, you can use the read method of the file object. Here's an example that reads the entire contents of a file into a string: file = open("example.txt", "r") contents = file.read() print(contents) file.close()



Explain how the read operation works assuming that the file to be read has the following contents:

File contents:
Line 1: This is line 1.
Line 2: This is line 2.
Line 3: This is line 3.

- The initial read position is at the beginning of the file.
- When you call the readline method on the file object for the first time, the read position advances to the end of the first line. In other words, the read position is now at the beginning of the second line.
- When you call the readline method again, the read position advances to the end of the second line, and so on, until you reach the end of the file.
- This way, the read position keeps advancing to the next line each time the readline method is called, allowing you to read the contents of the file line by line.

6.5 Closing a file

It is important to close a file after you are done with it to free up resources. You can do this by calling the close method of the file object.

It is recommended to use the with statement when working with files in Python, as it automatically closes the file for you, even if an exception is raised: with open("example.txt", "r") as file:

contents = file.read() print(contents)

Examples

He is an example of a Python program that creates a new file, writes some text to it, reads the contents of the file, and then closes the file:

```
# Open the file in write mode file =
open("example.txt", "w")

# Write some text to the file file.write("This is line 1.\n")
file.write("This is line 2.\n") file.write("This is line 3.\n")

# Close the file file.close()

# Open the file in read mode file =
open("example.txt", "r")

# Read and print the contents of the file contents =
file.read() print(contents)

# Close the file file.close()
```

When you run this program, it will create a new file named example.txt in the current directory, write the specified text to the file, read the contents of the file, and then print the contents to the console. The program then closes the file, releasing any resources it was using.

Note that the write method writes the text to the file, while the read method reads the contents of the file into a string. The close method closes the file and frees up any resources it was using. It's important to close the file when you're done working with it to ensure that any changes you made to the file are saved and that the file is not locked by the program, preventing other programs from accessing it.

The following example uses the readline method instead of the read method.

```
# Open the file in write mode file =
open("example.txt", "w")
```

```
# Write some text to the file file.write("This is line 1.\n")
file.write("This is line 2.\n") file.write("This is line 3.\n")

# Close the file file.close()

# Open the file in read mode file =
open("example.txt", "r")

# Read and print the contents of the file, line by line
line = file.readline()                    # read first line
line = file.readline()                    # read second line
line = file.readline()                    # read third line

# Close the file file.close()
```

In reality, a programmer doesn't know how many lines of texts a file contains but still want to read the file line by line. This can be achieved using the a loop; here is an example:

```
# Open the file in write mode file =
open("example.txt", "w")

# Write some text to the file file.write("This is line 1.\n")
file.write("This is line 2.\n") file.write("This is line 3.\n")

# Close the file file.close()

# Open the file in read mode file =
open("example.txt", "r")

# Read and print the contents of the file, line by line line = file.readline()
while line:
        print(line)     line =
    file.readline()
```

```
# Close the file file.close()
```

When you run this program, it will create a new file named example.txt in the current directory, write the specified text to the file, read the contents of the file line by line, and then print each line to the console. The program then closes the file, releasing any resources it was using.

Note that the readline method reads a single line of the file at a time, and returns an empty string when it reaches the end of the file. The while loop continues to read and print lines from the file until the readline method returns an empty string, indicating that the end of the file has been reached.

```
Output
This is line 1.
This is line 2. This
is line 3.
```

Each line of the file is printed on its own line, including the line break at the end of each line in the file. The line break is represented by the newline character '\n', which is automatically added to the end of each line when you write to the file, and is also preserved when you read the contents of the file using the readline method.

6.6 Concatenating a Newline to a String

In Python, you can concatenate a newline character to a string using the string literal \n. For example:

```
string = "This is a string" string_with_newline = string + "\n"
print(string_with_newline)

Output
This is a string
```

Reading a String and Stripping the Newline From It

You can strip the newline character from the end of a string using the strip method. For example:

```
string_with_newline = "This is a string\n" stripped_string =
string_with_newline.strip() print(stripped_string)
```

```
Output
 This is a string
```

So, the original string "This is a string\n" has had the newline character \n stripped from the end of it using the strip method, and the result is a string without a line break. So, the original string "This is a string\n" has had the newline character '\n' stripped from the end of it using the strip method, and the result is a string without a line break.

You can also strip specific characters from the end of a string using the rstrip method.

```
string_with_newline = "This is a string\n" stripped_string =
string_with_newline.rstrip("\n") print(stripped_string)
```

```
 Output
 This is a string
```

The rstrip method has stripped the newline character '\n' from the end of the string, resulting in a string without a line break.

Here is another example:

```
# This program reads the contents of the  #
philosophers.txt file one line at a time.
 def main():
        # Open a file named philosophers.txt.  infile =
       open('philosophers.txt', 'r')
```

```
    # Read three lines from the file  line1 =
    infile.readline()  line2 = infile.readline()
    line3 = infile.readline()

    # Strip the \n from each string.
    line1 = line1.rstrip('\n')  line2 =
    line2.rstrip('\n') line3 = line3.rstrip('\n')

    # Close the file.  infile.close()

    # Print the data that was read into memory.
    print(line1)  print(line2)
    print(line3)

# Call the main function.  if _ _name_ _
== '_ _main_ _':
main()
```

Output

John Locke

David Hume

Edmund Burke

- This program reads the contents of a file named philosophers.txt one line at a time. The program starts by opening the file in read mode using infile = open('philosophers.txt', 'r').
- Then, it uses the readline method to read three lines from the file, and assigns each line to a separate variable (line1, line2, and line3).
- Next, the program strips the newline characters (‘\n’) from each of the strings using the rstrip method, so that the contents of the file can be processed more easily.
- Finally, the program closes the file using infile.close(), and prints the contents of the three lines that were read from the file.

- The program uses the if __name__ == '__main__' idiom to ensure that the main function is only called when the program is executed as a standalone script, and not when the module is imported as a library.

6.7 Appending Data to an Existing File

To open an output file in append mode, use the 'a' mode. If the file already exists, it will not be erased. If the file does not exist, it will be created; when data is written to the file, it is appended to the end of the contents that are already there.

Example:

```
# Open the file in append mode with
open('file.txt', 'a') as f:
        # Write new data to the end of the file
        f.write('This is a new line of text.\n')

# Verify that the data was written to the file with open('file.txt', 'r') as
f:
        print(f.read())
```

In this example, the file 'file.txt' is opened in append mode using the with statement and the 'a' mode flag. The contents of the file are then appended with a new line of text using the write method. Finally, the file is read and its contents are printed to verify that the new data was written to the file correctly.

Note that when you use the write method to write data to a file, you need to include the line break character \n at the end of each line if you want to preserve the line breaks in the file.

6.8 Writing and Reading Numeric Data

Before writing, numbers must be converted to strings. A value is turned into a string by the built-in function str.

```
outfile.write(str(num1) + ‘\n’) infile =
open(‘number.txt’, ‘r’) # readline method
reads strings string_input = infile.readline()

# built-in function int converts string to an integer
value = int(string_input)
```

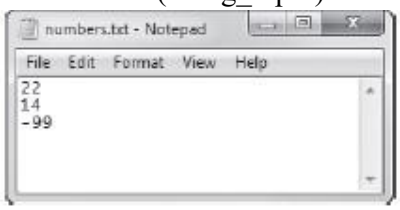


Figure 6-1 The numbers.txt file viewed in Notepad

Source: Gaddis (2021) page 318

Here is another example: # Open the file for reading with

```
open('numbers.txt', 'r') as file:     # Read the first line from
the file     line1 = file.readline()     # Read the second line
from the file     line2 = file.readline()     # Read the third line
from the file     line3 = file.readline()

# Strip the newline character from each line line1 =
line1.rstrip('\n') line2 = line2.rstrip('\n') line3 =
line3.rstrip('\n')

# Convert each line to a numeric value number1 =
float(line1) number2 = float(line2) number3 =
float(line3)

# Add the numbers together sum = number1
+ number2 + number3
```

```
# Print the sum of the numbers
print('The sum of the numbers is:', sum)
```

- This code will open the file numbers.txt in read mode and read the first, second, and third lines from the file. It will then strip the newline characters from each line using the rstrip method and convert each line to a numeric value using the float function.
- Finally, the code adds the numbers together and prints the sum of the numbers. Note that this code assumes that each line in the file contains a valid floating-point number, and will raise an error if this is not the case. You should add error handling code to handle cases where the contents of the file are not as expected.

6.9 Reading a File with a Loop and Detecting the End of the File

It allows reading a file's contents without being aware of how many items are contained therein. When the readline method tries to read beyond the file's end, it produces an empty string (' '). If utilizing a while loop, priming read is required to test the loop condition.

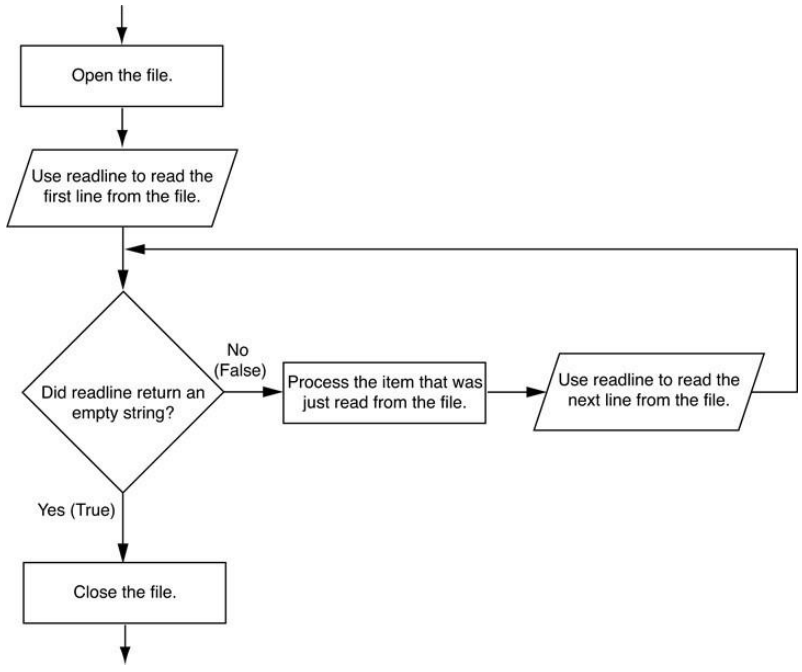


Figure 6-2 General logic for detecting the end of a file

Using Python's for Loop to Read Lines

A line of text is automatically read from the input file by the for loop. There is no special condition or testing needed and the priming read is not required. The loop automatically terminates whenever the file's end is reached.

```
for variable in file_object:
    statement
    statement       etc.
```

Example:

```
    # Open the file for reading with
    open('numbers.txt', 'r') as file:
      # Initialize a variable to keep track of the sum    sum = 0

      # Loop over each line in the file    for line in file:
            # Strip the newline character from the line       line = line.rstrip('\n')
            # Convert the line to a numeric value       number =
    float(line)       # Add the number to the sum       sum += number #
    Print the sum of the numbers print('The sum of the numbers is:',
    sum)
```

- This code will open the file numbers.txt in read mode and loop over each line in the file. For each line, it will strip the newline character from the line, convert the line to a numeric value using the float function, and add the number to a running total sum.
- Finally, the code will print the sum of the numbers. Note that this code assumes that each line in the file contains a valid floating-point number, and will raise an error if this is not the case. You should add error handling code to handle cases where the contents of the file are not as expected.

Refer to the prescribed textbook for more information and examples.



Review Questions

1. A file that data is read from is known as a(n) .

a. input file

b. output file

c. sequential access file

d. binary file

2. Before a file can be used by a program, it must be .

a. formatted

b. encrypted

c. closed

d. opened

3. This type of file contains data that has not been converted to text. a. text file

b. binary file

c. Unicode file

d. symbolic file

4. When working with this type of file, you access its data from the beginning of the file to the end of the file.

a. ordered access

b. binary access

c. direct access

d. sequential access

5. This is a small “holding section” in memory that many systems write data to before writing the data to a file.

a. buffer

b. variable

c. virtual file

d. temporary file

6. This marks the location of the next item that will be read from a file.

a. input position

b. delimiter

c. pointer

d. read position

7. When a file is opened in this mode, data will be written at the end of the file's existing contents.

a. output mode

b. append mode

c. backup mode

d. read-only mode

8. True/False

When you open a file that file already exists on the disk using the 'w' mode, the contents of the existing file will be erased.

9. True/False

The process of opening a file is only necessary with input files. Output files are automatically opened when data is written to them.

10. If an existing file is opened in append mode, what happens to the file's existing contents?

11. If a file does not exist and a program attempts to open it in append mode, what happens?

12. Assume a file containing a series of names (as strings) is named names.txt and exists on the computer's disk. Write a program that displays the number of names that are stored in the file. (Hint: Open the file and read every string stored in it. Use a variable to keep a count of the number of items that are read from the file.)

13. Write a program that writes a series of random numbers to a file. Each random number should be in the range of 1 through 500. The application should let the user specify how many random numbers the file will hold.

References

Gaddis, T. (2021). Starting out with Python. 5th Global Ed. United Kingdom: Pearson Education. ISBN: 9781292408637

Python.org. (2019). The Python Tutorial — Python 3.8.0 documentation. [online] Available at: https://docs.python.org/3/tutorial/.